Security without Trusted Third Parties: VRF-based Authentication with Short Authenticated Strings

Yanqi Gu, Stanislaw Jarecki, Phillip Nazarian, and Apurva Rai

University of California Irvine, USA,
{yanqig1,sjarecki,pnazaria,apurvr1}@uci.edu

Abstract. Message authentication (MA) in the Short Authenticated String (SAS) model, defined by Vaudenay [25], allows for authenticating *arbitrary messages* sent over an insecure channel as long as the sender can also transmit to the receiver a *short authenticated message*, e.g. $d = 20$ bits. The flagship application of SAS-MA is Authenticated Key Exchange (AKE) in the SAS model (SAS-AKE), which allows parties communicating over an insecure network to establish a secure channel without a prior source of trust except an ability to exchange d-bit authenticated strings. SAS-AKE is applicable e.g. for *device pairing*, i.e. creating secure channels between devices capable of displaying d-bit values, e.g. encoded as decimal strings, verified by a human operator, or for secure *messaging applications* like Signal or WhatsApp, where such short values can be read off by participants who trust each others' voices.

A string of works [25,20,15] showed light-weight SAS-MA schemes, using only symmetric-key crypto and 3 communication flows, which is optimal [25]. In [16] this was extended to group SAS-(M)MA, for (mutual) message authentication among any number of parties, using *two simultaneous flows*. We show a new two simultaneous flows SAS-(M)MA protocol, based on *Verifiable Random Functions* (VRF), with a novel property that the first flow, which consists of exchanging VRF public keys, can be *re-used in multiple SAS-MA instances*.

Moreover, instantiated with ECVRF, these keys have the same form $vk = g^{sk}$ as Diffie-Hellman keys exchanged in DH-based (A)KE protocols like X3DH. We show that X3DH keys can be re-used in our SAS-MA, implying SAS-AKE which adds a minimal overhead of a *single flow* to X3DH. Crucially, while X3DH is secure only if participants' public keys are certified by a shared source of trust, e.g. a Public Key Infrastructure (PKI) or a trusted Key Distribution Center (KDC) ran by Signal or WhatsApp, if X3DH is amended by our SAS-AKE then the established channel is secure even if PKI or KDC is compromised, assuming trust in user-assisted authentication of short d-bit strings.

1 Introduction

Message authentication in the SAS model. Serge Vaudenay [25] introduced the notion of a Message Authentication (MA) protocol in a Short Authenticated

String model (SAS).[1] A SAS-MA protocol allows for authentication of messages of arbitrary sizes, sent over an insecure channel, making use of an auxiliary channel connecting the sender and the receiver, which can be used to *authenticate short strings*, e.g. $d = 20$ bits. Assume that an adversary exercises complete control over the insecure channel, while the only restriction on the auxiliary channel is that the adversary cannot modify or inject messages on it, but it can eavesdrop and delay or drop them. Crucially, there is no other source of trust, i.e. no pre-shared keys or passwords, or a trusted third party, e.g. a PKI Certification Authority (CA) or a Key Distribution Service (KDC).

Following the 'MANA' protocol of [9], Vaudenay [25] showed a 3-flow SAS-MA with optimal security, i.e. receiver R cannot accept message m_R different from m_S sent by sender S except for probability $1/2^d$. It is optimal because $1/2^d$ is the collision probability even if $\mathtt{sas}_S$ and $\mathtt{sas}_R$ generated by $S(m_s)$ and $R(m_R)$ were random d-bit strings. Vaudenay also observed that 3 flows is an optimal round complexity for SAS-MA. Laur, Asokan, and Nyberg [14,15] and Pasini and Vaudenay [20] reduced SAS-MA assumptions to non-malleable commitments, and extended it to 3-flow SAS-MCA, i.e. message *cross*-authentication, where both parties authenticate a message to a counterparty, and both send out short authenticated strings. Laur and Pasini [16] extended these further to the *group* setting, enabling (mutual) message authentication (SAS-MMA) among a group of any $n \geq 2$ parties in *two simultaneous flows*.[2]

Authenticated key exchange in the SAS model. The primary practical application of SAS-MA is Authenticated Key Exchange (AKE) in the SAS model (SAS-AKE), i.e. to establish a secure communication channel between two (or more) parties based on their ability to exchange d-bit authenticated strings. Such SAS-AKE protocols have at least two important use-cases: First, they enable *device pairing*, i.e. establishing secure channels between devices which can display d-bit values, e.g. encoded as decimal or alphanumerical strings, whose equality can be verified by a human operator.[3] Secondly, SAS-AKE can be used to secure *messaging applications* like Signal or WhatsApp, where short verification values, encoded e.g. as strings in an alphabet chosen by the user, can be read off by participants who can verify the authenticity of each others' voices.[4] Specifically, SAS-AKE establishes secure channels without trust in any third party, e.g. a PKI Certification Authority (CA) or a Key Distribution Service (KDC).

Contribution 1: VRF-based SAS-(M)MA. We show a new SAS-MMA protocol, for an arbitary $n > 2$ number of parties, based on *Verifiable Random Functions* (VRF) [17]. Unlike prior SAS-(M/C)MA protocols, which require

[1] The conference version of this paper appeared in [12].

[2] For 2 parties a naive implementation of a 2 simultaneous flow protocol would use 4 messages, but the two flows from one party can be piggybacked, resulting in a 3 message protocol, matching the round-complexity lower-bound for SAS-MA.

[3] There are other implementations of SAS channels, relying on users to verify equality of e.g. beeping sequences, buzzing sequences, or blinking LED lights [3,11,24,23].

[4] In the context of audio or video interactions one can consider other encodings of d-bit values, e.g. as phrases in a user-chosen language or sequences of gestures.

very light-weight cryptography (a low-cost non-malleable commitment can use a cryptographic hash modeled as a Random Oracle), our SAS-MMA is based on a primitive whose practical instantiations are based on public-key crypto, e.g. the costs of ECVRF [10] are similar to a Diffie-Hellman key exchange. Our SAS-MMA uses 2 simultaneous flows and can be implemented in 3 flows in the 2-party case, exactly like the SAS-MMA of [16].

The novelty of our SAS-MMA is a property that the first flow, which consists of exchanging VRF public keys, can be *re-used in multiple SAS-MMA instances*. In particular, after a single exchange of the re-usable first flow, parties could run any number of subsequent SAS-MMA instances using only one simultaneous flow per instance. This, however, does not appear very useful (and has been modeled before in the regular message authentication context eg. [22]) because the same parties could have used the first SAS-MA to authenticate their signature public keys or an authenticated channel, and either way would allow them to authenticate future communication without resorting to the SAS-model authentication. However, the true benefit of our VRF-based SAS-MMA scheme is that there are many authentication scenarios in which participants keys are pre-distributed. (Crucially, these keys do not have to be trusted for the security of our scheme.) If these keys can be used as VRF keys then the message complexity overhead of SAS-MMA reduces to a single message sent by each party. Indeed, if the VRF in our SAS-MMA is instantiated with ECVRF, the VRF keys have the same form $vk = g^{sk}$ as Diffie-Hellman keys exchanged in e.g. DH-based AKE, which gives us hope that these same keys can be re-used for the VRF-based SAS-MMA.

Contribution 2: SAS-AKE from SAS-MMA integrated with X3DH. We show that the above intuition applies to some AKE's of interest. Specifically, we detail an extension of X3DH [6], an AKE protocol which is used in several messaging applications including WhatsApp and Signal, which integrates X3DH with our SAS-MMA protocol instantiated with ECVRF. X3DH is a variant of *3DH*, an AKE scheme which uses three instances of Diffie-Hellman to 'pair' long-term and one-time keys of the two parties, assuring perfect-forward secrecy [6]. X3DH amends each party's key set with semi-static keys, the KDC stores every party's long-term key together with their semi-static key and a set of one-time keys, and when some party inititates the X3DH protocol and requests their counterparty's keys from the KDC, it receives the counterparty's long-term key together with a semi-static key and the next unused one-time key.

Our X3DH+SAS-MMA integration proposal has several attractive performance features: First, it makes no changes to the key registration phase, and no changes to the KDC server operation. Second, the SAS-MMA computational overhead roughly matches the cost of X3DH itself: it requires 5 (multi)-exponentiations per party, compared to 5 made in X3DH. Third, its comunication overhead is only $3x/8$ Bytes piggybacked with the first X3DH message of both parties if it is instantiated with an x-bit elliptic curve, e.g. 96B using a 256-bit curve. The round complexity overhead of SAS-AKE over X3DH is effectively one message (from the responder Bob to the initiator Alice) because X3DH proper consists of a single message from Alice to Bob (in addition to

both parties retrieving counterparty's keys from the KDC). Responder Bob can compute its SAS value $\texttt{sas}_B$, and start transmitting it to Alice, e.g. reading the encoding of $\texttt{sas}_B$ in his voice, already in his first message, while the initiator Alice can compute her SAS value $\texttt{sas}_A$, and start transmitting it to Bob in her second message.

The point of the X3DH and SAS-MMA integration is to provide stronger security to the secure channels established by X3DH. Recall that X3DH requires trust in the KDC: If the KDC server is compromised and does not return true counterparty's keys to the requester (i.e. Bob's long-term key to Alice and Alice's long-term key to Bob) then X3DH by itself, like any AKE protocol, provides no security: The honest party who retrieves an adversarial key from the KDC will establish a secure channel with an adversary who created this key. If, however, X3DH is amended by SAS-MMA then the established channel will be secure *even if KDC is compromised*, under the assumption that the user-assisted authentication of the SAS transmission can be trusted.

Secondly, note that even if the KDC is trusted, the X3DH protocol is subject to an active attack if any party's long-term key is compromised (the attacker can play the initiator role on behalf of that party against any responder). However, the integration of X3DH with SAS-MMA ensures security in this case: SAS-MMA authenticates *all keys* used in a X3DH instance, including one-time keys, and the established session is secure even if any party's long-term key is compromised but their one-time key is not.

Roadmap. In Section 2 we define the syntax and security of VRF, the main tool we use in our SAS-MMA construction, and we recall ECVRF. In Section 3 we define the syntax and security of SAS-MMA. In Section 4 we present our VRF-based SAS-MMA protocol, and we argue its security. Finally, in Section 5 we specify our proposed integration of X3DH with our VRF-based SAS-MMA protocol. For the reason of space constraints we defer a few things to the Appendix, including game-based VRF security definitions in Appendix B and a formal definition of SAS-MMA correctness in Appendix A.

2 Cryptographic Preliminaries

In this section we overview Verifiable Random Functions (VRF), the main tool used in our SAS-MMA scheme $\mathsf{VrfAuth}$ of Section 4.

Notation. We use κ for the security parameter, $[n]$ for an integer range $\{1, ..., n\}$, and $\mathcal{F}[X, Y]$ for a set of functions from domain X to codomain Y. If S is a set, $x \leftarrow_\$ S$ denotes sampling x uniformly at random from S.

Computational Assumptions. Let $\mathbb{G}$ be a cyclic group of order p with generator g, and assume a (possibly deterministic) algorithm $\texttt{ParG}$ which generates group description $(\mathbb{G}, p, g)$ for each κ. Let CDH_g be a function s.t. $\mathsf{CDH}_g(g^a, g^b) = g^{ab}$ for all $a, b \in \mathbb{Z}_p$. We say that the *Computational Diffie-Hellman* (CDH) assumption holds in a group family defined by $\texttt{ParG}$ if for any efficient algorithm $\mathcal{A}$ there is a negligible function ϵ s.t. $\Pr[\mathcal{A}(g, p, g^a, g^b) = g^{ab} \,|\, (\mathbb{G}, p, g) \leftarrow$

$\mathtt{ParG}(1^\kappa), (a,b) \leftarrow_\$ \mathbb{Z}_p \times \mathbb{Z}_p] \leq \epsilon(\kappa)$. Let DDH_g be an oracle s.t. $\mathsf{DDH}_g(A,B,C)$ outputs 1 if $C = \mathsf{CDH}_g(A,B)$ and 0 otherwise. We say that the *Gap Diffie-Hellman* (GapDH) assumption holds for group family defined by $\mathtt{ParG}$ if the same holds even if $\mathcal{A}$ has access to oracle DDH_g, i.e. if $\Pr[\mathcal{A}^{\mathsf{DDH}_g(\cdot,\cdot,\cdot)}(g,p,g^a,g^b) = g^{ab} \mid (\mathbb{G},p,g) \leftarrow \mathtt{ParG}(1^\kappa), (a,b) \leftarrow_\$ \mathbb{Z}_p \times \mathbb{Z}_p] \leq \epsilon(\kappa)$.

Verifiable Random Function (VRF). A Verifiable Random Function (VRF), introduced by Micali, Rabin and Vadhan in [17], is a Pseudorandom Function (PRF) with the following additional properties: The key generation algorithm KG creates a PRF key sk together with a verification key vk which is a commitment to sk. The VRF evaluation algorithm $\mathsf{Eval}(sk,x)$ computes $z = \mathsf{PRF}_{sk}(x)$, where PRF is the underlying PRF function, together with a (non-interactive) *proof* π that z is a correctly computed output of function PRF_{sk} on input x for key sk committed in vk. The scheme comes with a verification algorithm Ver s.t. $\mathsf{Ver}(vk,x,z,\pi) = 1$ if $(z,\pi) \leftarrow \mathsf{Eval}(sk,x)$ for (sk,vk) output by KG. Algorithm KG will need some global parameters ppm generated by $\mathtt{ParG}(1^\kappa)$, but e.g. in the ECVRF scheme [10] one can use with our SAS-MMA protocol, the ppm parameter is the specification of a prime-order elliptic curve and associated hash functions, which are a fixed part of the specification of the algorithm.

We assume that the domain of the underlying PRF function PRF are arbitrary bitstrings and the range are bitstrings of length $p(\kappa)$ where p is some polynomial, i.e. $\mathsf{PRF}_{sk} : \{0,1\}^* \to \{0,1\}^{p(\kappa)}$. Formally, a VRF scheme is a tuple of efficient algorithms $\mathsf{VRF} = (\mathtt{ParG}, \mathsf{KG}, \mathsf{Eval}, \mathsf{Ver})$, with the following syntax:

- $\mathtt{ParG}(1^\kappa) \to \mathsf{ppm}$ is an algorithm which generates global parameters ppm.
- $\mathsf{KG}(\mathsf{ppm}) \to (sk, vk)$: A key generation algorithm which generates a PRF private key sk together with a commitment vk to this key.
- $\mathsf{Eval}(sk,x) \to (z,\pi)$: For any $x \in \{0,1\}^*$, algorithm Eval computes the PRF value $z = \mathsf{PRF}_{sk}(x)$ s.t. $z \in \{0,1\}^{p(\kappa)}$, together with a proof π.
- $\mathsf{Ver}(vk,x,z,\pi) \to 1/0$: An algorithm which, given proof π, verifies whether $z = \mathsf{PRF}_{sk}(x)$ for sk committed in vk.

The VRF scheme must satisfy three fundamental properties, which are *correctness/completeness*, *uniqueness/soundness*, and *(simulatable) pseudorandomness*. Correctness is self-explanatory, uniqueness means that it is infeasible to create a verification key vk for which one can prove that two different values are both correct VRF outputs on the same argument x. Pseudorandomness says that the z's output by Eval are indistinguishable from random values, i.e. that function PRF_{sk} is a PRF. We include formal definitions of these properties in Appendix B.

Universally Composable VRF. There are other properties which make VRFs easier to use. Peikert and Xu [21] introduced VRF *binding*, which asks that a single proof π can be used for at most one (vk,x,z) tuple, and VRF *non-malleability*, which is akin to *simulation soundness* of zero-knowledge proofs, i.e. it asks that adversarial VRFs are unique even if honest parties' oracles $\mathsf{Eval}_{sk}(\cdot)$ are replaced by random functions and a simulator. Another potentially useful property is *VRF pseudorandomness even for adversarially chosen keys.*

Ideal Functionality $\mathcal{F}^d_{\mathsf{VRF}}$:
Functionality $\mathcal{F}^d_{\mathsf{VRF}}$ interacts with parties P_i and an adversary $\mathcal{A}^*$. It maintains tables $T[\cdot,\cdot]$ and $P[\cdot]$ whose entries are initially set to $\perp$, and the following sets, all initially set to empty: S_{pk} for registered keys, S_{Eval} for VRF evaluations, and $\mathcal{C}$ for adversarial and compromised keys.

Key Compromise. On $(\mathsf{Key\text{-}Comp}, \mathsf{P}_i, \mathsf{kptr})$ from $\mathcal{A}^*$ do the following:
if $\exists vk$ s.t. $(\mathsf{P}_i, \mathsf{kptr}, vk) \in S_{pk}$ then $\mathcal{C} \leftarrow \mathcal{C} \cup \{vk\}$

Key Generation. On $(\mathsf{KeyGen}, \mathsf{kptr})$ from P_i s.t. $(\mathsf{P}_i, \mathsf{kptr}, \cdot) \notin S_{pk}$, hand $(\mathsf{KeyGen}, \mathsf{kptr}, \mathsf{P}_i)$ to $\mathcal{A}^*$, and on vk from $\mathcal{A}^*$ s.t. $(\cdot,\cdot,vk) \notin S_{pk}$ do the following:
$S_{pk} \leftarrow S_{pk} \cup \{(\mathsf{P}_i, \mathsf{kptr}, vk)\}$, return $(\mathsf{VerificationKey}, \mathsf{kptr}, \mathsf{P}_i, vk)$ to P_i

Malicious Key Generation. On $(\mathsf{MalKeyGen}, vk)$ from $\mathcal{A}^*$ s.t. $(\cdot,\cdot,vk) \notin S_{pk}$ do:
$S_{pk} \leftarrow S_{pk} \cup \{(\mathcal{A}^*, \perp, vk)\}$, $\mathcal{C} \leftarrow \mathcal{C} \cup \{vk\}$, return $(\mathsf{VerificationKey}, vk)$ to $\mathcal{A}^*$

VRF Evaluation and Proof. On $(\mathsf{Eval}, \mathsf{kptr}, x)$ from P_i where $\exists vk$ s.t. $(\mathsf{P}_i, \mathsf{kptr}, vk) \in S_{pk}$, hand $(\mathsf{Eval}, \mathsf{P}_i, \mathsf{kptr}, x)$ to $\mathcal{A}^*$, and on receiving π from $\mathcal{A}^*$ do:
1. assert $(P[\pi] = \perp \vee P[\pi] = (vk, x))$
2. if $T[vk,x] = \perp$ then set $z \leftarrow_{\$} \{0,1\}^d$ and $T[vk,x] \leftarrow (z, \{\pi\})$
   else set $(z, S) \leftarrow T[vk,x]$ and $T[vk,x] \leftarrow (z, S \cup \{\pi\})$
3. $S_{\mathsf{Eval}} \leftarrow S_{\mathsf{Eval}} \cup \{(vk, x, z)\}$, $P[\pi] \leftarrow (vk, x)$
4. return $(\mathsf{Evaluated}, \mathsf{kptr}, x, z, \pi)$ to P_i

Malicious VRF Evaluation. On $(\mathsf{MalEval}, vk, x)$ from $\mathcal{A}^*$ do the following:
1. if $(vk \in \mathcal{C} \wedge T[vk,x] = \perp)$ then set $z \leftarrow_{\$} \{0,1\}^d$ and $T[vk,x] \leftarrow (z, \varnothing)$
2. if $\exists (z, S)$ s.t. $T[vk,x] = (z, S)$ then return $(\mathsf{Evaluated}, vk, x, z)$ to $\mathcal{A}^*$

Verification. On $(\mathsf{Verify}, vk, x, z, \pi)$ from any party, P_i or $\mathcal{A}^*$, hand $(\mathsf{Verify}, vk, x, z, \pi, S_{\mathsf{Eval}})$ to $\mathcal{A}^*$, and on receiving ϕ from $\mathcal{A}^*$ do the following:
1. if $(\cdot,\cdot,vk) \in S_{pk}$ and $\exists (z, S)$ s.t. $T[vk,x] = (z, S)$ then:
   if $\pi \in S$ or $(\phi = 1 \wedge P[\pi] = \perp)$ then $f \leftarrow 1$
   (if $(\phi = 1 \wedge P[\pi] = \perp)$ also set $T[vk,x] \leftarrow (z, S \cup \{\pi\})$ and $P[\pi] \leftarrow (vk, x)$)
   in all other cases set $f \leftarrow 0$
2. return $(\mathsf{Verified}, vk, x, z, \pi, f)$ to the caller

Adversarial Leakage. On $(\mathsf{PastEvaluations})$ from $\mathcal{A}^*$ return S_{Eval} to $\mathcal{A}^*$

Fig. 1: Universally Composable VRF model $\mathcal{F}_{\mathsf{VRF}}$ [2].

All the above properties are captured by a single *Universally Composable* VRF definition introduced by David et al. [7] and refined in [1,2]. Defining VRF security in Canetti's framework of Universal Composability (UC) [4] consists of defining an ideal model called a VRF *functionality*, which specifies how the VRF scheme should ideally behave, and this specification implies e.g. that each key vk uniquely identifies a VRF instance, that the VRF outputs look random and are uniquely specified for each (vk, x) pair, etc. A VRF scheme which is shown to securely realize such functionality enables a modular analysis of a protocol that uses VRF's, like our SAS-MMA scheme $\mathsf{VrfAuth}$, because in the analysis

one can effectively replace all calls to the VRF scheme with the calls to the ideal VRF functionality, thus abstracting away a VRF implementation with a clear contract imposed by the ideal functionality.

Ideal VRF Functionality. We define the ideal VRF functionality $\mathcal{F}_{\mathsf{VRF}}$ in Figure 1, adopting the refinements in the UC VRF definition by Badertscher et al. [2], who showed that this functionality is realized under the CDH assumption in the Random Oracle Model (ROM) by ECVRF [10], see below. We state functionality $\mathcal{F}_{\mathsf{VRF}}$ using slightly different syntax than used in [2], e.g. we use an additional table $P[\cdot]$ to retrieve the pair (vk, x) corresponding to a given proof π, but it is an equivalent functionality.

An honest party P_i first interacts with $\mathcal{F}_{\mathsf{VRF}}$ by generating a key with the KeyGen command. Parties are allowed to initialize multiple VRF keys, so a key pointer kptr is used to distinguish between them. Whenever any P_i generates a key, the ideal adversary $\mathcal{A}^*$ picks the bitstring vk which represents it, and $\mathcal{F}_{\mathsf{VRF}}$ imposes no requirement on these verification keys except that they must be unique. Adversary $\mathcal{A}^*$ can use the Key-Comp interface to *compromise* any key vk generated by some P_i, which is tracked using set $\mathcal{C}$. $\mathcal{A}^*$ can also directly generate adversarial keys via MalKeyGen.

When an honest party evaluates VRF via the Eval interface, on some inputs (vk, x) which wasn't evaluated so far, functionality $\mathcal{F}_{\mathsf{VRF}}$ samples random z of the prescribed length $p(\kappa)$, which captures the pseudorandomness property of a VRF. Adversary $\mathcal{A}^*$ picks the bitstring π which represents the proof of correct VRF evaluation (whether old or new), and this bitstring must be either fresh or it is a bitstring which was previously registered as a proof for the same pair (vk, x). This captures simulatability, i.e. zero-knowledge, with the strengthening that $\mathcal{A}^*$ must generate π *before* learning the VRF function output z. It also captures the binding property, because a single proof bitstring π cannot be reused for two different (vk, x) pairs. However, note that every time VRF is evaluated on the same (vk, x) pair, $\mathcal{A}^*$ can specify a different proof π, which models VRFs where proofs are randomized, as is the case e.g. for ECVRF, see below. Adversary $\mathcal{A}^*$ can use the MalEval interface to evaluate VRF for any compromised key $vk \in \mathcal{C}$, and $\mathcal{F}_{\mathsf{VRF}}$ samples random output z for such evaluations as well, which captures the property that VRF outputs are pseudorandom even on adversarial keys. We note that $\mathcal{A}^*$ can also use MalEval or PastEvaluations interfaces to observe past VRF evaluations performed by honest parties, $vk \notin \mathcal{C}$.[5]

Functionality $\mathcal{F}_{\mathsf{VRF}}$ uses tables T, P and sets $S_{pk}, S_{\mathsf{Eval}}$ to keep a state: Set S_{pk} stores tuples $(\mathsf{P}_i, \mathsf{kptr}, vk)$ and $(\mathcal{A}^*, \perp, vk)$ corresponding to keys created by either honest parties or an adversary. Set S_{Eval} contains all (vk, x, z) tuples s.t. VRF was evaluated under key vk on input x and z was an output assigned to this evaluation. For each bitstring π which was ever used as a VRF proof, table $P[\pi]$ stores pair (vk, x) to which proof π was assigned. Finally, if $(vk, x, z) \in S_{\mathsf{Eval}}$ then $T[vk, x] = (z, S)$ where S might be empty, if vk is compromised and $\mathcal{A}^*$

[5] This leakage of VRF evaluations of honest parties would be a weakness of the ideal VRF model of [2] in applications that rely on VRF outputs to be private, e.g. by encrypting them, but in our application all VRF evaluations are public by default.

evaluates VRF on (vk, x) via the $\mathsf{MalEval}$ interface, but otherwise S contains all proof strings π which have been assigned to (vk, x), i.e. all π s.t. $P[\pi] = (vk, x)$.

Finally, $\mathcal{F}_{\mathsf{VRF}}$ verification interface Verify models VRF verification, and it reflects the soundness property of a VRF, because call $(\mathsf{Verify}, vk, x, z, \pi)$ succeeds only if z is the correct output assigned to (vk, x) at a prior VRF evaluation on this pair, i.e. if $T[vk, x] = (z, S)$ for some S. If the proof π being verified has already been recorded in set S stored in $T[vk, x]$, then successful verification is automatic. If, however, π is fresh, i.e. $P[\pi] = \perp$, then $\mathcal{A}^*$ has the choice, via bit ϕ, to either register π as a new valid proof for pair (vk, x), or to make the verification fail.[6] Finally, if VRF wasn't evaluated on (vk, x), or it was but z was not the output, or tuple (vk, x, z) is correct but π is registered for a different (vk, x) pair, then verification always fails.

To complete the comparison between $\mathcal{F}_{\mathsf{VRF}}$ and the game-based VRF syntax, observe that given VRF scheme $\mathsf{VRF} = (\mathtt{ParG}, \mathsf{KG}, \mathsf{Eval}, \mathsf{Ver})$ the interface which $\mathcal{F}_{\mathsf{VRF}}$ gives to the honest parties should be implemented as follows. First, we assume that $\mathtt{ParG}(1^\kappa)$ was executed, either by a trusted party or by anyone if $\mathtt{ParG}$ is deterministic, to generate public parameters ppm. Party P_i can then implement query $(\mathsf{KeyGen}, \mathsf{kptr})$ by sampling $(sk_{\mathsf{kptr}}, vk_{\mathsf{kptr}}) \leftarrow \mathsf{KG}(\mathsf{ppm})$, retaining $(\mathsf{kptr}, sk_{\mathsf{kptr}})$, and returning $(\mathsf{VerificationKey}, \mathsf{kptr}, \mathsf{P}_i, vk_{\mathsf{kptr}})$ to the environment. Party P_i implements query $(\mathsf{Eval}, \mathsf{kptr}, x)$ by retrieving sk_{kptr} associated with kptr, generating $(z, \pi) \leftarrow \mathsf{Eval}(sk_{\mathsf{kptr}}, x)$ and returning $(\mathsf{Evaluated}, \mathsf{kptr}, x, z, \pi)$. Finally, any party can implement query $(\mathsf{Verify}, vk, x, z, \pi)$ by returning the bits generated by $\mathsf{Ver}(vk, x, z, \pi)$.

Elliptic Curve VRF (ECVRF). The Elliptic Curve VRF was proposed in [19], as a low-cost VRF which can be implemented in any prime-order elliptic curve where the Computational Diffie-Hellman (CDH) assumption holds. ECVRF is used commercially e.g. in blockchain applications like Cardano and Algorand, and has been proposed for standardization via an IRTF draft [10]. Since one of our goals is to compose our SAS-MMA protocol instantiated with ECVRF with an Authenticated Key Exchange protocol $\mathsf{X3DH}$ used e.g. by Signal (see Section 5), and $\mathsf{X3DH}$ is customarily expressed using a multiplicative group notation, we adopt the same multiplicative group notation in the description of ECVRF below, instead of an additive notation customarily used for elliptic curve groups:

- $\mathtt{ParG}(1^\kappa)$ defines an elliptic curve group $\mathbb{G}$ of prime order p and generator g, with three hash functions modeled as Random Oracles, $H : \{0,1\}^* \to \mathbb{G}$, $H' : \mathbb{G} \to \{0,1\}^{p(\kappa)}$, and $H^* : \mathbb{G}^6 \to \mathbb{Z}_p$, and outputs $\mathsf{ppm} = (\mathbb{G}, p, g, H, H', H^*)$.
- $\mathsf{KG}(\mathsf{ppm})$ samples $sk \leftarrow_\$ \mathbb{Z}_p$ as the PRF private key together with a public key $vk = g^{sk}$, and outputs (sk, vk), with both implicitly containing ppm.

[6] A weakness of this model is that it doesn't imply proof consistency, i.e. if two parties use fresh π in a verification of a correct prior VRF evaluation point (vk, x, z), but π hasn't been registered as a correct proof for that tuple, then $\mathcal{A}^*$ can make the verification of the first party fail, but make the same verification performed by the second party succeed. We stress that security of our SAS-MMA application of VRF is *not* affected by this feature of the $\mathcal{F}_{\mathsf{VRF}}$ model of [2].

– $\mathsf{Eval}(sk, x)$ computes $h = H(x)$ and $w = h^{sk}$, and outputs (z, π) where the PRF value is set as $z = H'(w)$, and the proof as $\pi = (w, c, s)$, where (c, s) is a Fiat-Shamir NIZK proof for the statement that (g, vk, h, w) is a DH tuple, i.e. $c = H^*(g, h, vk, w, g^r, h^r)$ for $r \leftarrow_\$ \mathbb{Z}_p$ and $s = r - c \cdot sk \mod p$.
– $\mathsf{Ver}(vk, x, z, \pi)$ parses $(w, c, s) \leftarrow \pi$, sets $h = H(x)$, and outputs 1 if $z = H'(w)$ and $c = H^*(g, h, vk, w, vk^c \cdot g^s, w^c \cdot h^s)$ and 0 otherwise.

Theorem 1 ([2, Theorem 9.3]). ECVRF *UC-realizes* $\mathcal{F}_{\mathsf{VRF}}$ *in the random oracle model under the assumption that the* CDH *problem is hard in the prime order group* $\mathbb{G}$ *generated by* $\mathtt{ParG}(1^\kappa)$.

3 Mutual Message Authentication in the SAS Model

In this section we define the syntax and the security goals of a SAS-MMA scheme for groups of arbitrary size. SAS-MMA for groups was defined before by Laur and Pasini [16], but we need a new formal definition because our proposed SAS-MMA scheme has an unusual feature of re-use of long-term keys across SAS-MMA instances. Providing the formal definitions gives us also the opportunity to clarify the security game, and justify the modeling decisions we make.

Communication setting. MMA in the Short Authenticated String (SAS) model [20] considers a set of parties $\{\mathsf{P}_1, ..., \mathsf{P}_n\}$ who want to mutually authenticate a message, i.e. if P_i's input is denoted m_i, then each P_i wants to confirm that $m_j = m_i$ for all $j \in [n] \setminus \{i\}$. The parties communicate over an *insecure network* and have no pre-established source of trust, except for an additional communication resource of a *bandwidth-limited authenticated channel*. Namely, for some parameter d, each P_i can send out one d-bit message, denoted $\mathtt{sas}_i$, on the *Short Authenticated String* (SAS) *channel* associated with P_i. This channel is not secret, so the adversary learns $\mathtt{sas}_i$, but it preserves integrity and source authentication, i.e. the adversary can stop or delay the delivery of $\mathtt{sas}_i$ to any protocol participant, but it cannot modify this message or replace it with its own. One example of this setting is *device pairing*, where a set of devices wants to mutually authenticate a transcript of a (group) key agreement, and each device P_i can display a d-bit message, which a human can confirm as being all equal. In another application, n users can authenticate a group key agreement transcript by communicating a short d-bit message using their voice, where each user is assumed to recognize and trust the voices of all other intended participants. In this setting, a d-bit code message spoken by an owner of party P_i, and transmitted over an insecure network, implements a short authenticated string which the adversary is unable to forge.

SAS-MMA syntax. In contrast to prior SAS protocols, we define a SAS-MMA scheme as including a *pre-processing algorithm* Init, for each party to create a state it can reuse across multiple SAS-MMA protocol instances. We call the reusable message created by Init a *public key*, and the corresponding private state a *private key*, denoted resp. vk and sk. Formally, we define SAS-MMA scheme as a tuple of efficient algorithms $\Pi = (\mathsf{Init}, \mathsf{MsgGen}, \mathsf{SasGen}, \mathsf{Fin})$, parametrized by the SAS channel bandwidth d, with the following syntax:

- $\mathsf{Init}(1^\kappa) \to (sk, vk)$: A pre-processing algorithm each party uses to generate a private state sk and a "pre-message", a.k.a. a public key, vk.
- $\mathsf{MsgGen}(sk, \mathsf{VK}, m) \to pm$: An algorithm party P uses to generate a protocol message pm. P's inputs include P's private state sk, vector $\mathsf{VK} = (vk_1, ..., vk_n)$ of public keys of $n \geq 2$ of intended protocol participants (including P's own vk), and message m to be authenticated.
- $\mathsf{SasGen}(sk, \mathsf{VK}, m, \mathsf{PM}) \to \mathtt{sas}$: An algorithm party P uses to generate its d-bit SAS message $\mathtt{sas} \in \{0,1\}^d$. In addition to sk, VK, m the inputs include a vector PM of messages of all participants, including P's message.
- $\mathsf{Fin}(\mathtt{sas}, \mathsf{SAS}) \to 1/0$: An algorithm party P uses to verify a vector SAS of SAS messages of all protocol participants, including P. Party P needs to retain only its own SAS message $\mathtt{sas}$ to perform this verification.

  Note: If algorithm Fin is not described then we assume it is a *canonical* algorithm, which accepts if and only if vector SAS of received SAS messages contains n copies of message $\mathtt{sas}$. (All SAS-MMA schemes we know used a canonincal Fin algorithm.)

Since our proposed SAS-MMA protocol takes only a single simultaneous flow, assuming the participants pre-generate and exchange their public keys vk, our protocol syntax is simplified to a non-interactive algorithm MsgGen. Furthermore, in our protocol, algorithm SasGen does not need any state retained from executing MsgGen beyond message pm generated by MsgGen (which is assumed included in the input vector PM), and likewise algorithm Fin does not need any state beyond value $\mathtt{sas}$ generated by SasGen. However, this syntax can be extended to accommodate protocols which are interactive and/or where parties pass a private state to algorithms SasGen and Fin.

Correctness. Assuming the canonical algorithm Fin, a SAS-MMA scheme is intended to be used as follows: Let $S = \{\mathsf{P}_1, ..., \mathsf{P}_n\}$ for any $n \geq 2$ be a set of parties s.t. each P_i generated a key pair $(sk_i, vk_i) \leftarrow \mathsf{Init}(1^\kappa)$. For any m, if each P_i generates a protocol message $pm_i \leftarrow \mathsf{MsgGen}(sk_i, \mathsf{VK}, m)$ for $\mathsf{VK} = (vk_1, ..., vk_n)$, and a SAS message $\mathtt{sas}_i \leftarrow \mathsf{SasGen}(sk_i, \mathsf{VK}, m, \mathsf{PM})$ for $\mathsf{PM} = (pm_1, ..., pm_n)$, and if SAS messages $\mathtt{sas}_i$ are exchanged between the participating parties without adversarial interference, then all parties accept. We include a formal definition of this correctness notion in Appendix A.

Security. We model security of SAS-MMA scheme Π via a security game $\mathsf{Game}^{\mathsf{MMAsec}}_{\mathcal{A},\Pi}$, shown in Figure 2. In this game the adversary $\mathcal{A}$ can initialize an arbitrary number of keys per party and it can ask any party P_i to execute the SAS-MMA protocol using any key it holds, on any input message m, and for an arbitrary set of intended protocol participants. Since the protocol executes over an insecure network, all messages from P_i's counterparties, including the vector of precomputation messages VK and protocol messages PM, are decided by $\mathcal{A}$ as well. The only thing that the adversary cannot control are the SAS messages $\mathtt{sas}_j$ created by any P_i's intended counterparty P_j, which is reflected in the adversarial *win condition* in game $\mathsf{Game}^{\mathsf{MMAsec}}_{\mathcal{A},\Pi}$, as we explain below.

The only special requirement we impose is that each party never uses the same key vk for the same input m, i.e. that pair (vk, m) is *fresh*. Party P_i

$\underline{\mathsf{Game}^{\mathsf{MMAsec}}_{\mathcal{A},\Pi}(\kappa)}$:
$\forall i, \mathsf{sid}$ set $\mathsf{phase}[i,\mathsf{sid}]$ to 0, and $\mathsf{m}[i,\mathsf{sid}], \mathsf{key}[i,\mathsf{sid}], \mathsf{group}[i,\mathsf{sid}], \mathsf{state}[i,\mathsf{sid}], \mathtt{sas}[i,\mathsf{sid}]$ to $\perp$; set $\mathsf{used}, \mathsf{klist}, \mathsf{atk}, \mathcal{C}_{vk}, \mathcal{C}_{\mathtt{sas}}$ to $\varnothing$
$(i, j, \mathsf{sid}, \mathcal{G}) \leftarrow \mathcal{A}^{\Pi_0,\Pi_1,\Pi_2,\mathsf{FixTarget},\mathsf{Key\text{-}Comp},\mathsf{SAS\text{-}Comp}}(1^\kappa)$
assert $(\mathsf{atk} = (\mathsf{sid}, \mathcal{G}) \wedge \mathsf{phase}[i,\mathsf{sid}] = 2 \wedge \mathsf{phase}[j,\mathsf{sid}] = 2 \wedge \mathsf{group}[i,\mathsf{sid}] = \mathcal{G})$
if $j \in \mathcal{G} \ \wedge \ \mathsf{m}[i,\mathsf{sid}] \neq \mathsf{m}[j,\mathsf{sid}] \ \wedge i, j \notin \mathcal{C}_{\mathtt{sas}} \ \wedge \ \mathsf{key}[i,\mathsf{sid}] \notin \mathcal{C}_{vk} \wedge \mathsf{key}[j,\mathsf{sid}] \notin \mathcal{C}_{vk} \ \wedge$
$\wedge \ \forall k \in \mathcal{G} \setminus \mathcal{C}_{\mathtt{sas}} \ \mathtt{sas}[i,\mathsf{sid}] = \mathtt{sas}[k,\mathsf{sid}]$ then return 1 else return 0

$\underline{\Pi_0(i)}$:
$(sk, vk) \leftarrow \mathsf{Init}(1^\kappa)$, $\mathsf{klist} \leftarrow \mathsf{klist} \cup \{(i, sk, vk)\}$
return vk

$\underline{\Pi_1(i, \mathsf{sid}, vk, ind, \mathsf{VK}, m, \mathcal{G})}$:
assert $(\mathsf{phase}[i,\mathsf{sid}] = 0 \wedge i \in \mathcal{G} \wedge |\mathsf{VK}| = |\mathcal{G}| \wedge \mathsf{VK}[ind] = vk)$
assert $((i, sk, vk) \in \mathsf{klist} \wedge (vk, m) \notin \mathsf{used})$
$pm \leftarrow \mathsf{MsgGen}(sk, \mathsf{VK}, m)$
$\mathsf{phase}[i,\mathsf{sid}] \leftarrow 1$, $\mathsf{m}[i,\mathsf{sid}] \leftarrow m$, $\mathsf{key}[i,\mathsf{sid}] \leftarrow vk$, $\mathsf{group}[i,\mathsf{sid}] \leftarrow \mathcal{G}$
$\mathsf{used} \leftarrow \mathsf{used} \cup \{(vk, m)\}$, $\mathsf{state}[i,\mathsf{sid}] \leftarrow (sk, |\mathcal{G}|, ind, \mathsf{VK}, pm)$
return pm

$\underline{\Pi_2(i, \mathsf{sid}, \mathsf{PM})}$:
assert $\mathsf{phase}[i,\mathsf{sid}] = 1$
$(sk, n, ind, \mathsf{VK}, pm) \leftarrow \mathsf{state}[i,\mathsf{sid}]$
$m \leftarrow \mathsf{m}[i,\mathsf{sid}]$
assert $(|\mathsf{PM}| = n \wedge \mathsf{PM}[ind] = pm)$
$\mathtt{sas}[i,\mathsf{sid}] \leftarrow \mathsf{SasGen}(sk, \mathsf{VK}, m, \mathsf{PM})$
$\mathsf{phase}[i,\mathsf{sid}] \leftarrow 2$
return $\mathtt{sas}[i,\mathsf{sid}]$

$\underline{\mathsf{SAS\text{-}Comp}(i)}$:
$\mathcal{C}_{\mathtt{sas}} \leftarrow \mathcal{C}_{\mathtt{sas}} \cup \{i\}$

$\underline{\mathsf{FixTarget}(\mathsf{sid}, \mathcal{G})}$:
assert $\mathsf{atk} = \varnothing$
assert $\forall t \in \mathcal{G} \ \ \mathsf{phase}[t,\mathsf{sid}] = 0$
$\mathsf{atk} \leftarrow (\mathsf{sid}, \mathcal{G})$

$\underline{\mathsf{Key\text{-}Comp}(vk)}$:
assert $\exists i \ (i, sk, vk) \in \mathsf{klist}$
$\mathcal{C}_{vk} \leftarrow \mathcal{C}_{vk} \cup \{vk\}$
return sk

Fig. 2: Security game for SAS-MMA scheme $\Pi = (\mathsf{Init}, \mathsf{MsgGen}, \mathsf{SasGen})$

can assure this in several ways, e.g. by using one-time keys vk, or by running SAS-MMA in an application which enforces that message m is always unique, e.g. because m is a transcript of key agreement (KA) protocol, or because an application prepends m with a fresh random nonce.

Game $\mathsf{Game}^{\mathsf{MMAsec}}_{\mathcal{A},\Pi}$ models adversary $\mathcal{A}$'s interaction with honest parties via three oracles: Calling $\Pi_0(i)$ makes P_i generate a new key pair (sk, vk), which is added to P_i's list of keys $\mathsf{klist}[i]$. Calling $\Pi_1(i, \mathsf{sid}, vk, ind, \mathsf{VK}, m, \mathcal{G})$ asks P_i to run a SAS-MMA instance using key $vk \in \mathsf{klist}[i]$, on inputs VK, m and a group $\mathcal{G}$ of P_i's intended counterparties. This call specifies also P_i's position ind in the ordering of that group, and the game ensures that vector VK contains key vk

at position *ind*. Finally, the game ensures that pair (vk, m) is *fresh* (see above), by rejecting if it is on a list used of previously encountered pairs. After $\mathcal{A}$ calls Π_1 on some (i, sid), the game sets $\mathsf{phase}[i, \mathsf{sid}]$ to 1, which allows $\mathcal{A}$ to then call $\Pi_2(i, \mathsf{sid}, \mathsf{PM})$ so P_i generates a SAS message $\mathtt{sas}[i, \mathsf{sid}]$ on that session, for any PM chosen by $\mathcal{A}$, with the only constraint that P_i's own message pm appears in PM at position *ind*.

Throughout the game the adversary can adaptively compromise any long-term key, via oracle call $\mathsf{Key\text{-}Comp}(vk)$, and adaptively compromise party P_i's SAS channel, via oracle call $\mathsf{SAS\text{-}Comp}(i)$. The former leaks the corresponding private key sk, while the latter gives the adversary an ability to write on P_i's SAS channel, which models e.g. the adversary learning to emulate P_i's voice in the SAS-MMA application where the SAS channel is implemented by a party reading its SAS messages in their own voice.

We say that $\mathcal{A}$ *wins* the security game, i.e. the game returns bit 1, if the adversary outputs $(i, j, \mathsf{sid}, \mathcal{G})$ s.t. (1) $j \in \mathcal{G}$, i.e. P_j is in the intended group $\mathcal{G}$ of parties P_i wanted to authenticate on that session, (2) $m_i \neq m_j$ where m_i and m_j were respectively P_i's and P_j's inputs on that session, (3) P_i and P_j's SAS channels were not compromised, and keys vk_i, vk_j used by resp. $\mathsf{P}_i, \mathsf{P}_j$ on that session were also not compromised (see a discussion on the effects of key and SAS compromise below), and (4) P_i accepted on the SAS-MMA session tagged by sid, and in particular all SAS messages $\mathtt{sas}[i, \mathsf{sid}] = \mathtt{sas}[k, \mathsf{sid}]$ for all $j \in \mathcal{G}$ s.t. P_k's SAS channel is not compromised. (We restrict the attack condition in this way because if P_k's SAS channel is compromised then the adversary can freely generate SAS messages on behalf of this party.)

Definition 1. *A SAS-MMA scheme Π is* secure *if for all efficient algorithms $\mathcal{A}$ there exists a negligible function ϵ s.t.* $\Pr[1 \leftarrow \mathsf{Game}^{\mathsf{MMAsec}}_{\mathcal{A},\Pi}(1^\kappa)] \leq 1/2^d + \epsilon(\kappa)$.

Security Model: Further Considerations.

Session identifiers. SAS-MMA session identifier sid plays an important role in our communication model. In particular, it crucially influences the final 'adversarial win' condition that defines an attack on the SAS-MMA scheme. Note that the win definition requires that for all parties P_k which are in $\mathcal{G}$, the set of parties P_i wanted to communicate with in the attacked session, but whose SAS channels were not compromised, i.e. for all $k \in \mathcal{G} \setminus \mathcal{C}_{\mathtt{sas}}$, we require that $\mathtt{sas}[i, \mathsf{sid}] = \mathtt{sas}[k, \mathsf{sid}]$, i.e. that the $\mathtt{sas}$ value P_k sent *on the session marked with the same* sid matches the $\mathtt{sas}$ value P_i expects. This condition effectively bounds the SAS messages that are pertinent to the attack to a single session per each intended participant. In particular, the adversary cannot "reroute" a SAS message which P_j sent on another session to the sid-identified session of P_j. We believe that this modeling choice is justified in the SAS application scenarios we target, i.e. for authentication of secure channel establishment in a messaging application, where the SAS channel has 'liveness' i.e. the listener knows when the sender is actually online and speaking his/her SAS message, and users are unlikely to engage in concurrent executions of SAS authentication.

By contrast, in Vaudeney's SAS-MA model [25] the SAS messages of each party are treated as if they are written to a whiteboard, and the adversary can then redirect these messages to *any* recipient at *any* point. Such 'whiteboard' treatment of SAS values implies that q instances of each party populate their SAS whiteboards with q values. Using the birthday bound, the base-line attack probability is then close to 1 for $q = 2^{d/2}$, which is not very meaningful for small d values. To better reflect real-world applications one can introduce "t-limited concurrency", by limiting the SAS whiteboard sizes to only the last t values, in which case the base-line attack probability would change to $t^2/2^d$. This would complicate our model, but we believe that our VRF-based scheme would still be secure under such notion. We note that the prior SAS-MMA model of Laur and Pasini [16] implicitly makes the same modeling choice as we do regarding session-binding of SAS messages. (Doing otherwise would allow the above attack, which is not reflected in the security bounds argued in [16].)

Choosing Attack Target. The adversary in our security notion is not fully adaptive in one aspect, namely it has to commit to the instance of the SAS-MMA protocol it wants to attack before that instance executes. Specifically, using oracle call $\mathsf{FixTarget}(\mathsf{sid}, \mathcal{G})$, it has to commit to the identifier sid and a group $\mathcal{G}$ of the (intended) participants of the attacked SAS-MMA instance, and it has to do this before any party P_i for $i \in \mathcal{G}$ engages in an instance indexed by sid. However, it does not have to specify the indexes $i, j \in \mathcal{G}$ of the group members which are directly involved in the attack described above. Intuitively, the optimal security SAS-model authentication can provide is akin to a coin-toss: Assume that in each SAS-MMA instance the d-bit SAS string, which honest participants output and which they also expect to see from others, is a perfectly random bitstring. Assume also that P_i and P_j intend to authenticate each other on a SAS-MMA instance, but the two parties run on respective inputs $m_i \neq m_j$, and the adversary isolates these parties, i.e. both interact only with an adversary $\mathcal{A}$. Note that even in this case, since the SAS strings $\mathtt{sas}_i$ and $\mathtt{sas}_j$ these two parties output are random d-bit strings, there is still $1/2^d$ chance that $\mathtt{sas}_i = \mathtt{sas}_j$. Perhaps counter-intuitively, this chance does not grow if there are $n \geq 2$ protocol participants: No matter the size of the intended group, the adversary wins only if it partitions the group into input-consistent subgroups (i.e. all parties within a partition run on the same m input) and the $\mathtt{sas}$ values in all subgroups are nevertheless all the same. (In particular, the maximum win probability of $1/2^d$ requires that the group is split into only two such subgroups.) However, if there are q protocol instances and the adversary attacks each one in this way, it would have $\leq q/2^d$ probability that the attack on *some* session succeeds, and since 2^d is small, this would not be a meaningless bound. Our notion avoids this "attack probability accumulation" by committing the adversary to the attacked session, and captures the optimal bound of $1/2^d$ attack probability.

Effects of Key Compromise. Recall that our model allows adaptive compromise of keys and SAS channels. However, we define a successful attack as involving pair of parties $\mathsf{P}_i, \mathsf{P}_j$ s.t. the keys vk_i, vk_j they use on the attacked session are *not compromised*. If algorithms MsgGen and SasGen of the SAS-MMA scheme are

deterministic then this restriction is necessary because of the following attack. Assume $n=2$ and $\mathcal{G}=\{i,j\}$, and an adversary who picks (sk^*, vk^*), starts P_j on any m_j and $\mathsf{VK}=(vk^*, vk_j)$ and runs the protocol on (sk^*, m_j) interacting with P_j until P_j outputs $\mathtt{sas}_j$. The adversary then uses sk_i and sk^* to locally run P_i's execution on $\mathsf{VK}'=(vk_i, vk^*)$ for any candidate message $m_i \neq m_j$, and tests if $\mathtt{sas}_i$ which P_i would output satisfies $\mathtt{sas}_i=\mathtt{sas}_j$. After expected 2^d attempts the adversary finds such m_i, and wins with probability 1 by asking the real P_i to execute on it. The leakage of P_j's private state sk_j leads to a similar attack, but done in the opposite order, i.e. the adversary first asks P_i to run on some m_i, learns $\mathtt{sas}_i$, and then uses sk_j to brute-force search for $m_j \neq m_i$ which leads P_j to output $\mathtt{sas}_j=\mathtt{sas}_i$.

We note that in our VRF-based SAS-MMA scheme algorithm MsgGen is randomized but its 'SAS contribution' is a deterministic function of sk and m, so it still falls prey to the same attack, hence we must restrict the model by giving up on sessions involving compromised keys.[7] We leave as an open question if any round-minimal SAS-MMA can use randomness to remain secure in spite of key compromise. We note also that the attack works only if the key is compromised *before* it is used on the attacked session, and our security notion can be strengthened to allow key compromise after the session terminates.

Effects of SAS Channel Compromise. Our attack condition also requires that the SAS channels of P_i and P_j are *not compromised*. If P_j's SAS channel is compromised, e.g. the adversary trains an AI model to forge P_j's voice, then the adversary can win with probability 1 by observing P_i's output $\mathtt{sas}_i$ and then injecting $\mathtt{sas}_j=\mathtt{sas}_i$ into P_j's compromised SAS channel. If P_i's SAS channel is compromised then the attack is more subtle but also more limited. Consider a session with n parties whose SAS channels are all compromised except for P_j, and let each party run on a different message. The attacker can win with probability $(n-1)/2^d$ by running each party in isolation (using adversarial keys) until each outputs its SAS message. If there exists $i \neq j$ s.t. $\mathtt{sas}_i=\mathtt{sas}_j$ then the adversary can fake the SAS channels for all $i \notin \{i,j\}$ to match $\mathtt{sas}_i$, and P_i will accept even though $m_i \neq m_j$. Note that for $n=2$ this probability is still $1/2^d$, and indeed if we support only 2-party SAS-MMA then the security holds even if P_i's SAS channel is compromised.

Freshness. As mentioned above, our definition only guarantees security in the case that no party ever runs twice on the same input (vk, m). This restriction puts some limitation on key reuse, but it is easy to satisfy in applications, e.g. if each party can inject a random nonce into message m. For example, in the X3DH application of SAS-MMA, see Section 5, message m includes an X3DH transcript, which is guaranteed to be fresh.

Group divergence. Our security model does not require that P_j involved in the attack ran the session indexed by sid with the same target group $\mathcal{G}$ which was assumed by P_i. Indeed, set $\mathcal{G}'$ of counterparties assumed by P_j on that session

[7] In Section 5.1 we show how our VRF-based SAS-MMA can be strenghtened so a party can use two (or more) keys and remain secure unless both are compromised.

indexed might not even include P_i. This "group divergence" might not be relevant in two-party mutual authentication, but for larger groups it means that P_i and P_j might have different views of the group of intended protocol participants, but as long as P_i accepted while assuming that P_j participated, and P_j and P_i ran on mismatched inputs, i.e. $m_i \neq m_j$, this would count as an adversarial win.

Note on dynamic groups. Our SAS-MMA model assumes that each P_i starts the protocol on a target group $\mathcal{G}$ of n participants, and P_i can abort if any $\mathsf{P}_j \in \mathcal{G}$ 'drops out'. (This is indeed the case in both our SAS-MMA protocol and in SAS-MMA of Laur and Pasini [16].) It would be preferable if SAS-MMA was robust to participants' failures, but it appears challenging to assure security in such scenario. In particular, in both SAS-MMA of [16] and of ours, if the adversary could remove any subset of parties from the initial group, it would have $O(2^n)$ choices for the SAS value output by P_i, and the attack probability would be $\approx 2^n/2^d$. Even if we wanted to tolerate only c dropped parties, the attack probability would be $\approx n^c/2^d$, which would not offer much security.

4 SAS-MMA from Verifiable Random Functions

We present our VRF-based SAS-MMA protocol, called VrfAuth, in Figure 3. In the first protocol round, i.e. Init, each party picks a VRF verification key vk. In the second round, i.e. MsgGen, each party evaluates their VRF on the mutual message m, sending out the d-bit VRF output z and associated proof π. Finally, to generate an SAS string, algorithm SasGen verifies all the received (z_t, π_t) pairs, and the d-bit string `sas` is an XOR of all z_t's. The finalization algorithm Fin is canonical, i.e. it accepts if all parties generate the same SAS message.

At a high level, VrfAuth can be seen as following a similar structure to previous SAS-MMA protocols such as that of Laur and Pasini [16]. In that protocol, each party first sends a commitment to a `sas` contribution for the message m, and these commitments are then opened in the second round. In VrfAuth, a VRF verification key vk acts as a commitment to a function mapping any future message m to a `sas` contribution z. The pseudorandomness and soundness of the function are analogous to the hiding (no one can predict the `sas` value) and binding (no one can change their `sas` contribution) properties of a commitment, respectively. Crucially, our usage of a VRF allows us to remove message m from the first round, which allows for reuse of vk across multiple protocol executions, and it allows vk to be defined before m is known (as in our integrated X3DH + SAS-MMA application, see Section 5).

Efficiency. Almost all prior SAS-M(M/C)A schemes, e.g. [25], [15], [20] and [16] were based solely on commitments, and therefore could be realized using only cryptographic hash functions. (One exception is [13], which use CCA-secure public key encryption, e.g. RSA OAEP, as part of a 3-flow SAS-AKE protocol.) By contrast, our protocol Π_{VrfAuth} uses VRF's whose (bandwidth-)efficient instantiations use public key cryptography. For example, if we instantiate VRF with ECVRF, each party performs 3 exponentiations in MsgGen and $2(n-1)$ multi-exponentiations (on two bases) in SasGen (because ECVRF Eval and Ver costs are

$\mathsf{VRF} = (\mathtt{ParG}, \mathsf{KG}, \mathsf{Eval}, \mathsf{Ver})$ is a Verifiable Random Function with range $\{0,1\}^d$.
Assume global parameters ppm generated as $\mathsf{ppm} \leftarrow \mathtt{ParG}(1^\kappa)$.
(Note that in ECVRF [19] parameters ppm can be verified by each participant and do not need to be generated by a trusted entity.)

$\underline{\mathsf{Init}(\mathsf{ppm})}$:
$(sk, vk) \leftarrow \mathsf{KG}(\mathsf{ppm})$
return (sk, vk)

$\underline{\mathsf{MsgGen}(sk, \mathsf{VK}, m)}$:
$(z, \pi) \leftarrow \mathsf{Eval}(sk, m)$
return $pm = (z, \pi)$

$\underline{\mathsf{SasGen}(sk, \mathsf{VK}, m, \mathsf{PM})}$:
$\{vk_1, \ldots, vk_n\} \leftarrow \mathsf{VK}$, $\{(z_1, \pi_1), \ldots, (z_n, \pi_n)\} \leftarrow \mathsf{PM}$
if $\forall t \in [n]$ $\mathsf{Ver}(vk_t, m, z_t, \pi_t)$ then return $\mathtt{sas} = \bigoplus_{t\in[n]} z_t$ else abort

$\underline{\mathsf{Fin}(\mathtt{sas}, \mathsf{SAS})}$:
$(\mathtt{sas}_1, \ldots, \mathtt{sas}_n) \leftarrow \mathsf{SAS}$
if $\forall t \in [n]$ $\mathtt{sas}_t = \mathtt{sas}$ then return 1 else return 0

Fig. 3: SAS-MMA scheme VrfAuth: $\Pi_{\mathsf{VrfAuth}} = (\mathsf{Init}, \mathsf{MsgGen}, \mathsf{SasGen}, \mathsf{Fin})$

respectively 3 exp's and 2 multi-exp's), a total of $2n+1$ (multi-)exponentiations, which is 5 (multi-)exp's in the two-party setting.[8]

We note that if $n > 2$ and the VRF scheme is ECVRF then n intances of VRF verification in step SasGen could be batched. In ECVRF the verification step consists of verifying a (non-interactive) ZK proof of discrete logarithm equality between $z = \mathsf{PRF}_{sk}(x) = (\mathsf{H}(x))^{sk}$ and $vk = g^{sk}$, and such proofs can be batch-verified, using the techniques of e.g. [18]. (Indeed, [2] show that an ECVRF variant which allows for batch verification remains a secure UC VRF.)

4.1 Security Analysis

Theorem 2. *SAS-MMA scheme* VrfAuth*, Figure 3, is* secure *if scheme* VRF *it uses realizes the UC VRF functionality* $\mathcal{F}_{\mathsf{VRF}}$*, Figure 1.*

Proof. We argue that SAS-MMA scheme VrfAuth is *secure* through a sequence of transformations. Throughout the transformations we fix the adversarial algorithm $\mathcal{A}_0$, and we start from G_0, see Figure 4, which is identical to the SAS-MMA security experiment $\mathsf{Game}^{\mathsf{MMAsec}}_{\mathcal{A}_0,\Pi}(\kappa)$ for $\Pi = \Pi_{\mathsf{VrfAuth}}$. Let $\epsilon_0 = \Pr[1 \leftarrow \mathsf{G}_0]$. Since G_0 is identical to the SAS-MMA security experiment for scheme VrfAuth, our goal is to show that $\epsilon_0 \leq 1/2^d + \epsilon(\kappa)$.

Game G_0 (real world): G_0 is the security experiment $\mathsf{Game}^{\mathsf{MMAsec}}_{\mathcal{A}_0,\Pi}(\kappa)$, see Fig. 2, for Π instantiated as Π_{VrfAuth}. To make it easier to verify, in Figure 4 we

[8] This is roughly the same cost as Signal's X3DH, which takes 4 (initiator) or 5 (responder) exponentiations, hence integrating our SAS-MMA with Signal's X3DH, as in Section 5, roughly doubles the computational cost for each party.

$\forall i, \mathsf{sid}$ set $\mathsf{phase}[i,\mathsf{sid}]$ to 0, and $\mathsf{m}[i,\mathsf{sid}], \mathsf{key}[i,\mathsf{sid}], \mathsf{group}[i,\mathsf{sid}], \mathsf{state}[i,\mathsf{sid}], \mathtt{sas}[i,\mathsf{sid}]$ to $\perp$; set $\mathsf{used}, \mathsf{klist}, \mathsf{atk}, \mathcal{C}_{vk}, \mathcal{C}_{\mathsf{sas}}$ to $\varnothing$

$(i, j, \mathsf{sid}, \mathcal{G}) \leftarrow \mathcal{A}_0{}^{\Pi_0,\Pi_1,\Pi_2,\mathsf{FixTarget},\mathsf{Key\text{-}Comp},\mathsf{SAS\text{-}Comp}}(1^\kappa)$

assert $(\mathsf{atk} = (\mathsf{sid}, \mathcal{G}) \wedge \mathsf{phase}[i,\mathsf{sid}] = 2 \wedge \mathsf{phase}[j,\mathsf{sid}] = 2 \wedge \mathsf{group}[i,\mathsf{sid}] = \mathcal{G})$

if $j \in \mathcal{G} \ \wedge \ \mathsf{m}[i,\mathsf{sid}] \neq \mathsf{m}[j,\mathsf{sid}] \ \wedge i, j \notin \mathcal{C}_{\mathsf{sas}} \ \wedge \ \mathsf{key}[i,\mathsf{sid}] \notin \mathcal{C}_{vk} \wedge \mathsf{key}[j,\mathsf{sid}] \notin \mathcal{C}_{vk} \ \wedge$
$\wedge \ \forall k \in \mathcal{G} \setminus \mathcal{C}_{\mathsf{sas}} \ \mathtt{sas}[i,\mathsf{sid}] = \mathtt{sas}[k,\mathsf{sid}]$ then return 1 else return 0

$\underline{\Pi_0(i)}$:
$(sk, vk) \leftarrow \mathsf{KG}(1^\kappa)$, $\mathsf{klist} \leftarrow \mathsf{klist} \cup \{(i, sk, vk)\}$
return vk

$\underline{\Pi_1(i, \mathsf{sid}, vk, ind, \mathsf{VK}, m, \mathcal{G})}$:
assert $(\mathsf{phase}[i,\mathsf{sid}] = 0) \wedge (i \in \mathcal{G}) \wedge (|\mathsf{VK}| = |\mathcal{G}|) \wedge (\mathsf{VK}[ind] = vk)$
assert $((i, sk, vk) \in \mathsf{klist}) \wedge ((vk, m) \notin \mathsf{used})$
$(z, \pi) \leftarrow \mathsf{Eval}_{sk}(m)$
$\mathsf{phase}[i,\mathsf{sid}] \leftarrow 1$, $\mathsf{m}[i,\mathsf{sid}] \leftarrow m$, $\mathsf{key}[i,\mathsf{sid}] \leftarrow vk$, $\mathsf{group}[i,\mathsf{sid}] \leftarrow \mathcal{G}$
$\mathsf{used} \leftarrow \mathsf{used} \cup \{(vk, m)\}$, $\mathsf{state}[i,\mathsf{sid}] \leftarrow (sk, |\mathcal{G}|, ind, \mathsf{VK}, (z, \pi))$
return (z, π)

$\underline{\Pi_2(i, \mathsf{sid}, \mathsf{PM})}$:
assert $\mathsf{phase}[i,\mathsf{sid}] = 1$
$(sk, n, ind, \mathsf{VK}, (z, \pi)) \leftarrow \mathsf{state}[i,\mathsf{sid}]$, $m \leftarrow \mathsf{m}[i,\mathsf{sid}]$
assert $(|\mathsf{PM}| = n) \wedge (\mathsf{PM}[ind] = (z, \pi))$
$(vk_1, \ldots, vk_n) \leftarrow \mathsf{VK}$, $((z_1, \pi_1), \ldots, (z_n, \pi_n)) \leftarrow \mathsf{PM}$
assert $\forall t \in [n]$ $\mathsf{Ver}_{vk_t}(m, z_t, \pi_t)$
$\mathtt{sas}[i,\mathsf{sid}] = \bigoplus_{t\in[n]} z_t$
$\mathsf{phase}[i,\mathsf{sid}] \leftarrow 2$
return $\mathtt{sas}[i,\mathsf{sid}]$

$\underline{\mathsf{FixTarget}(\mathsf{sid}, \mathcal{G})}$:
assert $\mathsf{atk} = \varnothing$
assert $\forall t \in \mathcal{G} \ \ \mathsf{phase}[t,\mathsf{sid}] = 0$
$\mathsf{atk} \leftarrow (\mathsf{sid}, \mathcal{G})$

$\underline{\mathsf{Key\text{-}Comp}(vk)}$:
assert $\exists i \ (i, sk, vk) \in \mathsf{klist}$
$\mathcal{C}_{vk} \leftarrow \mathcal{C}_{vk} \cup \{vk\}$
return sk

$\underline{\mathsf{SAS\text{-}Comp}(i)}$:
$\mathcal{C}_{\mathsf{sas}} \leftarrow \mathcal{C}_{\mathsf{sas}} \cup \{i\}$

Fig. 4: Game G_0: The security game $\mathsf{Game}^{\mathsf{MMAsec}}_{\mathcal{A}_0,\Pi}$, Fig. 2, instantiated with the SAS-MMA scheme Π_{VrfAuth} of Fig. 3. In yellow we show the code specific to scheme Π_{VrfAuth} and in blue we highlight VRF calls within that code.

use the standard font for the code that comes from the generic security game $\mathsf{Game}^{\mathsf{MMAsec}}_{\mathcal{A}_0,\Pi}$, see Fig. 2, and we use yellow and blue to mark the code that pertains to the specific SAS-MMA scheme Π_{VrfAuth}, where the latter color is used whenever Π_{VrfAuth}'s code contains calls to the VRF scheme VRF.

$\forall i, \mathsf{sid}$ set $\mathsf{phase}[i,\mathsf{sid}]$ to 0, and $\mathsf{m}[i,\mathsf{sid}], \mathsf{key}[i,\mathsf{sid}], \mathsf{group}[i,\mathsf{sid}], \mathsf{state}[i,\mathsf{sid}], \mathsf{sas}[i,\mathsf{sid}]$, $T[\cdot,\cdot]$ to $\perp$; set $\mathsf{used}, \mathsf{klist}, \mathsf{atk}, \mathcal{C}_{vk}, \mathcal{C}_{\mathsf{sas}}, S_\pi$ to $\varnothing$

$(i, j, \mathsf{sid}, \mathcal{G}) \leftarrow \mathcal{A}_1{}^{\Pi_0,\Pi_1,\Pi_2,\mathsf{MalKeyGen},\mathsf{MalEval},\mathsf{FixTarget},\mathsf{Key\text{-}Comp},\mathsf{SAS\text{-}Comp}}(1^\kappa)$

assert $(\mathsf{atk} = (\mathsf{sid}, \mathcal{G}) \wedge \mathsf{phase}[i,\mathsf{sid}] = 2 \wedge \mathsf{phase}[j,\mathsf{sid}] = 2 \wedge \mathsf{group}[i,\mathsf{sid}] = \mathcal{G})$

if $j \in \mathcal{G} \ \wedge \ \mathsf{m}[i,\mathsf{sid}] \neq \mathsf{m}[j,\mathsf{sid}] \ \wedge i, j \notin \mathcal{C}_{\mathsf{sas}} \ \wedge \ \mathsf{key}[i,\mathsf{sid}] \notin \mathcal{C}_{vk} \wedge \mathsf{key}[j,\mathsf{sid}] \notin \mathcal{C}_{vk} \ \wedge$
$\wedge \ \forall k \in \mathcal{G} \setminus \mathcal{C}_{\mathsf{sas}} \ \mathsf{sas}[i,\mathsf{sid}] = \mathsf{sas}[k,\mathsf{sid}]$ then return 1 else return 0

$\Pi_0(i, \mathsf{kptr}, vk)$:

assert $((i, \mathsf{kptr}, \cdot) \notin \mathsf{klist}) \wedge ((\cdot, \cdot, vk) \notin \mathsf{klist})$
$\mathsf{klist} \leftarrow \mathsf{klist} \cup \{(i, \mathsf{kptr}, vk)\}$

$\Pi_1(i, \mathsf{sid}, vk, ind, \mathsf{VK}, m, \mathcal{G}, \pi)$:

assert $(\mathsf{phase}[i,\mathsf{sid}] = 0) \wedge (i \in \mathcal{G}) \wedge (|\mathsf{VK}| = |\mathcal{G}|) \wedge (\mathsf{VK}[ind] = vk)$
assert $((i, \cdot, vk) \in \mathsf{klist}) \wedge (\pi \notin S_\pi) \wedge ((vk, m) \notin \mathsf{used})$
if $T[vk, m] = \perp$ then $z \leftarrow_\$ \{0,1\}^d$, $T[vk, m] \leftarrow (z, \{\pi\})$
$S_\pi \leftarrow S_\pi \cup \{\pi\}$, $\mathsf{phase}[i,\mathsf{sid}] \leftarrow 1$, $\mathsf{m}[i,\mathsf{sid}] \leftarrow m$, $\mathsf{key}[i,\mathsf{sid}] \leftarrow vk$, $\mathsf{group}[i,\mathsf{sid}] \leftarrow \mathcal{G}$
$\mathsf{used} \leftarrow \mathsf{used} \cup \{(vk, m)\}$, $\mathsf{state}[i,\mathsf{sid}] \leftarrow (|\mathcal{G}|, ind, \mathsf{VK}, (z, \pi))$
return (z, π)

$\Pi_2(i, \mathsf{sid}, \mathsf{PM}, \boldsymbol{\phi})$:

assert $\mathsf{phase}[i,\mathsf{sid}] = 1$
$(n, ind, \mathsf{VK}, (z, \pi)) \leftarrow \mathsf{state}[i,\mathsf{sid}]$, $m \leftarrow \mathsf{m}[i,\mathsf{sid}]$
assert $(|\mathsf{PM}| = n) \wedge (\mathsf{PM}[ind] = (z, \pi))$
$(vk_1, \ldots, vk_n) \leftarrow \mathsf{VK}$, $((z_1, \pi_1), \ldots, (z_n, \pi_n)) \leftarrow \mathsf{PM}$, $(\phi_1, \ldots, \phi_n) \leftarrow \boldsymbol{\phi}$
assert $\forall t \in [n] \ \ \exists (z_t, S_t) \ T[vk_t, m] = (z_t, S_t) \wedge (\pi_t \in S_t \vee (\phi_t = 1 \wedge \pi_t \notin S_\pi))$
$\forall t \in [n]$ if $(\phi_t = 1 \wedge \pi_t \notin S_\pi)$ then $T[vk_t, m] \leftarrow (z_t, S_t \cup \{\pi_t\})$
$\mathsf{sas}[i,\mathsf{sid}] \leftarrow \bigoplus_{t \in [n]} z_t$, $\mathsf{phase}[i,\mathsf{sid}] \leftarrow 2$, return $\mathsf{sas}[i,\mathsf{sid}]$

$\mathsf{MalKeyGen}(vk)$:

if $(\cdot, \cdot, vk) \notin \mathsf{klist}$ then
  $\mathsf{klist} \leftarrow \mathsf{klist} \cup \{(\mathcal{A}_1, \cdot, vk)\}$
  $\mathcal{C}_{vk} \leftarrow \mathcal{C}_{vk} \cup \{vk\}$

$\mathsf{MalEval}(vk, m)$:

if $(vk \in \mathcal{C}_{vk}) \wedge (T[vk, m] = \perp)$
  then $z \leftarrow_\$ \{0,1\}^d$, $T[vk, m] \leftarrow (z, \varnothing)$
if $T[vk, m] = (z, S) \neq \perp$ then return z

$\mathsf{FixTarget}(\mathsf{sid}, \mathcal{G})$:

assert $\mathsf{atk} = \varnothing$
assert $\forall t \in \mathcal{G} \ \ \mathsf{phase}[t,\mathsf{sid}] = 0$
$\mathsf{atk} \leftarrow (\mathsf{sid}, \mathcal{G})$

$\mathsf{Key\text{-}Comp}(vk)$:

assert $\exists (i, \mathsf{kptr}, vk) \in \mathsf{klist}$
$\mathcal{C}_{vk} \leftarrow \mathcal{C}_{vk} \cup \{vk\}$

$\mathsf{SAS\text{-}Comp}(i)$:

$\mathcal{C}_{\mathsf{sas}} \leftarrow \mathcal{C}_{\mathsf{sas}} \cup \{i\}$

Fig. 5: Game G_1: Replacing VRF calls from G_0 with $\mathcal{F}_{\mathsf{VRF}}$ code.

Game G_1 (implanting $\mathcal{F}_{\mathsf{VRF}}$): In G_1 we replace the real-world VRF scheme acting in G_0 with the ideal-world functionality $\mathcal{F}_{\mathsf{VRF}}$. In Figure 5, which shows G_1, all the calls to the VRF scheme made by G_0, denoted in blue in Fig. 4, are replaced by the code of the ideal functionality $\mathcal{F}_{\mathsf{VRF}}$ (see 1), denoted in green in Fig. 5. Since the adversary in the ideal-world (of VRF) has additional interfaces, namely malicious key generation and VRF evaluation via $(\mathsf{MalKeyGen}, vk)$ and $(\mathsf{MalEval}, vk, x)$, we add these interfaces in G_1. (However, we omit interface PastEvaluations because, see Fig. 1, it provides the set of all prior VRF evaluation tuples, but in the context of G_1, all VRF evaluations performed by honest parties are already known to the adversary.) Note that apart from changing blue-colored code fragments in G_0 to green-colored fragments in G_1, the remaining code of the two games is identical.

Since adversarial interfaces change between G_0 and G_1, the second game involves a formally different adversarial algorithm, denoted $\mathcal{A}_1$. Indeed, the transformation from G_0 to G_1 is justified by the assumption that scheme VRF realizes the UC VRF functionality $\mathcal{F}_{\mathsf{VRF}}$. Technically, what this fact implies is that for every efficient environment algorithm $\mathcal{Z}$ and adversary $\mathcal{A}_0$, there exists an efficient algorithm *simulator* $\mathcal{S}$ s.t. $\mathcal{Z}$'s view of an interaction with $\mathcal{A}_0$ and honest parties running scheme VRF, is computationally indistinguishable from $\mathcal{Z}$'s view of an interaction with the simulator $\mathcal{S}$ and honest parties who interact with the ideal VRF functionality $\mathcal{F}_{\mathsf{VRF}}$. Define $\mathcal{Z}$ as an algorithm which runs all the code of G_0 shown in Fig. 4, including adversary $\mathcal{A}_0$ as a subroutine run by $\mathcal{Z}$, except for the blue-colored fragments which correspond to honest parties P_i's making calls to VRF. Using this notation, G_0 is an interaction between $\mathcal{Z}$ using $\mathcal{A}_0$ as a subroutine, with honest P_i's running scheme VRF. By the assumption that VRF realizes $\mathcal{F}_{\mathsf{VRF}}$, there exists a simulator $\mathcal{S}$ s.t. this interaction is indistinguishable from an interaction of the same $\mathcal{Z}$ with $\mathcal{S}$ and honest P_i's interacting with $\mathcal{F}_{\mathsf{VRF}}$. The transformation between G_0 and G_1 is an instance of this: (1) adversary $\mathcal{A}_1$ used as subroutine in G_1 is a composition of adversary $\mathcal{A}_0$ and simulator $\mathcal{S}$, (2) the green-colored fragments of G_1 correspond to $\mathcal{F}_{\mathsf{VRF}}$ processing P_i's calls (that's the green-colored code in oracles Π_0, Π_1, Π_2 in Fig. 5) and $\mathcal{A}_1$'s direct calls (via oracles MalKeyGen, MalEval, and Key-Comp), and (3) the remaining code of G_1 is the environment $\mathcal{Z}$. By the assumption that VRF securely realizes $\mathcal{F}_{\mathsf{VRF}}$, it follows that there is a negligible function $\varepsilon_{\mathsf{VRF}}$ s.t. if $\epsilon_1 = \Pr[1 \leftarrow \mathsf{G}_1]$ then $\epsilon_0 \leq \epsilon_1 + \varepsilon_{\mathsf{VRF}}(\kappa)$.

Note that $\mathcal{A}_1$'s interfaces to oracles Π_0, Π_1, Π_2 in G_1 are different than $\mathcal{A}_0$'s interfaces to these oracles in G_0. In the real world, honest party P_i does not need any additional inputs to run VRF.KG, hence the only input $\mathcal{A}_0$ provides to Π_0 in G_0 is P_i's identifier i. However, in the ideal-world when party P_i generates a key via KeyGen interface of $\mathcal{F}_{\mathsf{VRF}}$ (see Fig. 1) it should supply a key identifier kptr. Further, $\mathcal{F}_{\mathsf{VRF}}$ reacts to $(\mathsf{KeyGen}, \mathsf{kptr})$ by sending the message $(\mathsf{KeyGen}, \mathsf{kptr}, \mathsf{P}_i)$ to adversary $\mathcal{A}_1$, who plays the role of the ideal-world adversary $\mathcal{A}^*$ of Fig. 1, asking $\mathcal{A}_1$ to supply a fresh public key vk. In G_1 we short-cut this interaction by asking $\mathcal{A}_1$ to supply tuple (i, kptr, vk) as input to Π_0.

$\mathcal{F}_{\mathsf{VRF}}$ on query $(\mathsf{Eval}, \mathsf{kptr}, x)$ from P_i for $(\mathsf{P}_i, \mathsf{kptr}, vk) \in S_{pk}$ hands this information to the ideal adversary who then supplies the proof π. Since the only oracle that triggers the $(\mathsf{Eval}, ...)$ query to $\mathcal{F}_{\mathsf{VRF}}$ is Π_1, and it does so in response to the adversary's query to Π_1, we short-cut this interaction as well, and ask $\mathcal{A}_1$ to hand the proof π along with all the other inputs at the time $\mathcal{A}_1$ queries Π_1. Similarly, $\mathcal{F}_{\mathsf{VRF}}$ on call $(\mathsf{Verify}, vk, x, z, \pi)$ from any party hands all these inputs to the ideal adversary $\mathcal{A}^*$ who supplies a bit ϕ. Since the only oracle that triggers a call $(\mathsf{Verify}, ...)$ is Π_2, and it does so in response to the adversary's query to Π_2, we short-cut this interaction as well and ask $\mathcal{A}_1$ to provide bit ϕ at the time $\mathcal{A}_1$ calls Π_2. Since Π_2 invokes $(\mathsf{Verify}, ...)$ for all $t \in [n]$, we ask $\mathcal{A}_1$ to supply a vector $\boldsymbol{\phi} = (\phi_1, ..., \phi_n)$ of these bits.

Notation in G_1 versus in $\mathcal{F}_{\mathsf{VRF}}$. The green code of G_1 corresponds to the code of $\mathcal{F}_{\mathsf{VRF}}$ implanted in the places where G_0 contains VRF commands KG, Eval, and Ver. However, this code in G_1 differs from how $\mathcal{F}_{\mathsf{VRF}}$ implements these calls, see Figure 1, in some notational choices which aim to simplify G_1 notation. First, $\mathcal{F}_{\mathsf{VRF}}$ uses set S_{pk} for keeping track of triples $(\mathsf{P}_i, \mathsf{kptr}, vk)$ denoting that P_i created public key vk with a local index kptr, while G_1 uses klist, inherited from G_0, for the same tuples but denoted (i, kptr, vk), because in the SAS-MMA security games it is more convenient for us to use indexes i as shortcut for party identities P_i. Also, $\mathcal{F}_{\mathsf{VRF}}$ keeps a table P s.t. $P[\pi]$ is either $\perp$, if π hasn't occured anywhere yet, or $P[\pi] = (vk, x)$ if π is associated with pair (vk, x). In G_1 we simplify this by keeping only a set S_π of proofs π which has occured in the system. The reason for this simplification is that $\mathcal{F}_{\mathsf{VRF}}$ allows for VRF's whose proofs might or might not be randomized (moreover, in the second case these proofs might or might not be randomizable by the adversary). To handle the case of deterministic proofs, $\mathcal{F}_{\mathsf{VRF}}$ allows that if Eval is executed on the same (vk, x) pair on which it was evaluated before then the simulator can provide the same proof π which has occured before. Hence $\mathcal{F}_{\mathsf{VRF}}$ handles Eval on (vk, m) by checking not only if π occured before but also whether $P[\pi] = (vk, m)$. However, in G_1 oracle Π_1 will reject if $(vk, m) \in \mathsf{used}$, i.e. if P_i performed Eval on the same (vk, m) pair before, hence G_1 does not have to check for the case that π has been used but on the same (vk, m) pair. $\mathcal{F}_{\mathsf{VRF}}$ uses the same check if $P[\pi] = \perp$ in handling Ver queries, and here too in G_1 we replace them by a check if $\pi \in S_\pi$.

Game G_2 (eliminating proofs): In G_2 we simplify G_1 based on the observation that the adversary can always predict whether their G_1 oracle calls will violate an "assert" statement and cause an abort, and that the adversary gains nothing by doing so. Therefore, G_2 removes the checks for proof validity from Π_2 and instead simply assumes that correct z values and proofs have been forwarded from Π_1. Proofs consequently play no role in G_2, which allows them to be removed from Π_1 as well. We define a new adversary $\mathcal{A}_2$ who runs $\mathcal{A}_1$ as a subroutine, internally performs the assert checks that have been removed from G_2, and only forwards those Π_1 and Π_2 calls that would not have violated those checks (with π, PM, and $\boldsymbol{\phi}$ excised from the calls in order to match the G_2 interface). $\mathcal{A}_2$ succeeds in G_2 iff $\mathcal{A}_1$ succeeds in G_1, i.e. $\epsilon_1 = \epsilon_2$, where we define $\epsilon_2 = \Pr[1 \leftarrow \mathsf{G}_2]$.

$\forall i, \mathsf{sid}$ set $\mathsf{phase}[i,\mathsf{sid}]$ to 0, and $\mathsf{m}[i,\mathsf{sid}], \mathsf{key}[i,\mathsf{sid}], \mathsf{group}[i,\mathsf{sid}], \mathsf{state}[i,\mathsf{sid}], \mathtt{sas}[i,\mathsf{sid}]$, $T[\cdot,\cdot]$ to $\perp$; set $\mathsf{used}, \mathsf{klist}, \mathsf{atk}, \mathcal{C}_{vk}, \mathcal{C}_{\mathtt{sas}}$ to $\varnothing$

$(i, j, \mathsf{sid}, \mathcal{G}) \leftarrow \mathcal{A}_2{}^{\Pi_0,\Pi_1,\Pi_2,\mathsf{MalKeyGen},\mathsf{MalEval},\mathsf{FixTarget},\mathsf{Key\text{-}Comp},\mathsf{SAS\text{-}Comp}}(1^\kappa)$

assert $(\mathsf{atk} = (\mathsf{sid}, \mathcal{G}) \wedge \mathsf{phase}[i,\mathsf{sid}] = 2 \wedge \mathsf{phase}[j,\mathsf{sid}] = 2 \wedge \mathsf{group}[i,\mathsf{sid}] = \mathcal{G})$

if $j \in \mathcal{G} \ \wedge \ \mathsf{m}[i,\mathsf{sid}] \neq \mathsf{m}[j,\mathsf{sid}] \ \wedge i, j \notin \mathcal{C}_{\mathtt{sas}} \ \wedge \ \mathsf{key}[i,\mathsf{sid}] \notin \mathcal{C}_{vk} \wedge \mathsf{key}[j,\mathsf{sid}] \notin \mathcal{C}_{vk} \ \wedge$
$\wedge \ \forall k \in \mathcal{G} \setminus \mathcal{C}_{\mathtt{sas}} \ \mathtt{sas}[i,\mathsf{sid}] = \mathtt{sas}[k,\mathsf{sid}]$ then return 1 else return 0

$\underline{\Pi_0(i, \mathsf{kptr}, vk)}$:
assert $((i, \mathsf{kptr}, \cdot) \notin \mathsf{klist}) \wedge ((\cdot, \cdot, vk) \notin \mathsf{klist})$
$\mathsf{klist} \leftarrow \mathsf{klist} \cup \{(i, \mathsf{kptr}, vk)\}$

$\underline{\Pi_1(i, \mathsf{sid}, vk, ind, \mathsf{VK}, m, \mathcal{G})}$:
assert $(\mathsf{phase}[i,\mathsf{sid}] = 0) \wedge (i \in \mathcal{G}) \wedge (|\mathsf{VK}| = |\mathcal{G}|) \wedge (\mathsf{VK}[ind] = vk)$
assert $((i, \cdot, vk) \in \mathsf{klist}) \wedge ((vk, m) \notin \mathsf{used})$
if $T[vk, m] = \perp$ then $z \leftarrow_\$ \{0,1\}^d$, $T[vk, m] \leftarrow z$
$\mathsf{phase}[i,\mathsf{sid}] \leftarrow 1$, $\mathsf{m}[i,\mathsf{sid}] \leftarrow m$, $\mathsf{key}[i,\mathsf{sid}] \leftarrow vk$, $\mathsf{group}[i,\mathsf{sid}] \leftarrow \mathcal{G}$
$\mathsf{used} \leftarrow \mathsf{used} \cup \{(vk, m)\}$, $\mathsf{state}[i,\mathsf{sid}] \leftarrow (|\mathcal{G}|, \mathsf{VK})$, return z

$\underline{\Pi_2(i, \mathsf{sid})}$:
assert $\mathsf{phase}[i,\mathsf{sid}] = 1$
$(n, \mathsf{VK}) \leftarrow \mathsf{state}[i,\mathsf{sid}], m \leftarrow \mathsf{m}[i,\mathsf{sid}], (vk_1, \ldots, vk_n) \leftarrow \mathsf{VK}$
assert $\forall t \in [n]\ T[vk_t, m] \neq \perp$
$\mathtt{sas}[i,\mathsf{sid}] \leftarrow \bigoplus_{t\in[n]} T[vk_t, m]$, $\mathsf{phase}[i,\mathsf{sid}] \leftarrow 2$, return $\mathtt{sas}[i,\mathsf{sid}]$

$\underline{\mathsf{MalKeyGen}(vk)}$:
if $(\cdot, \cdot, vk) \notin \mathsf{klist}$
  $\mathsf{klist} \leftarrow \mathsf{klist} \cup \{(\mathcal{A}_2, \cdot, vk)\}$
  $\mathcal{C}_{vk} \leftarrow \mathcal{C}_{vk} \cup \{vk\}$

$\underline{\mathsf{MalEval}(vk, m)}$:
if $(vk \in \mathcal{C}_{vk}) \wedge (T[vk, m] = \perp)$
  then $z \leftarrow_\$ \{0,1\}^d$, $T[vk, m] \leftarrow z$
if $T[vk, m] = z \neq \perp$ then return z

$\underline{\mathsf{FixTarget}(\mathsf{sid}, \mathcal{G})}$:
assert $\mathsf{atk} = \varnothing$
assert $\forall t \in \mathcal{G}\ \ \mathsf{phase}[t,\mathsf{sid}] = 0$
$\mathsf{atk} \leftarrow (\mathsf{sid}, \mathcal{G})$

$\underline{\mathsf{Key\text{-}Comp}(vk)}$:
assert $\exists (i, \mathsf{kptr}, vk) \in \mathsf{klist}$
$\mathcal{C}_{vk} \leftarrow \mathcal{C}_{vk} \cup \{vk\}$

$\underline{\mathsf{SAS\text{-}Comp}(i)}$:
$\mathcal{C}_{\mathtt{sas}} \leftarrow \mathcal{C}_{\mathtt{sas}} \cup \{i\}$

Fig. 6: Game G_2: Removing proofs from G_1, changes marked in yellow

It remains to upper-bound the adversary's probability of success in G_2. For the attacked session $\mathsf{atk} = (\mathsf{sid}, \mathcal{G})$ and attacked parties $i, j \in \mathcal{G}$, consider the two relevant Π_1 oracle calls $\Pi_1(i, \mathsf{sid}, vk_i, ind_i, \mathsf{VK}_i, m_i, \mathcal{G})$ and $\Pi_1(j, \mathsf{sid}, vk_j, ind_j, \mathsf{VK}_j, m_j, \mathcal{G}_j)$. FixTarget enforces that this sid was selected for attack before both of those Π_1 calls, and at the time of the Π_1 calls (VK_i, m_i) and (VK_j, m_j) commited the adversary to the $T[\cdot,\cdot]$ positions that would determine $\mathtt{sas}[i,\mathsf{sid}]$ and $\mathtt{sas}[j,\mathsf{sid}]$, respectively. $T[\cdot,\cdot]$ values are sampled uniformly at random by the game. Since the win condition specifies that $vk_i, vk_j \notin \mathcal{C}_{vk}$ and the Π_1 oracle enforces that $(vk_i, m_i), (vk_j, m_j) \notin \mathsf{used}$, the particular values $T[vk_i, m_i]$ and

$T[vk_j, m_j]$ were not yet sampled before the time of the respective Π_1 oracle calls. Observe lastly that the game's win condition requires $m_i \neq m_j$, which implies that $T[vk_i, m_i]$ is in the expression for $\mathsf{sas}[i, \mathsf{sid}]$ but not in the expression for $\mathsf{sas}[j, \mathsf{sid}]$ (and vice versa for $T[vk_j, m_j]$). It follows by a one-time pad argument that $\mathsf{sas}[i, \mathsf{sid}]$ and $\mathsf{sas}[j, \mathsf{sid}]$ are uniformly random, independent, and unpredictable to the adversary at the time of FixTarget, so the probability that they collide is 2^{-d}.

The adversary is still free to choose which $i, j \in \mathcal{G} \setminus \mathcal{C}_{\mathsf{sas}}$ to attack after FixTarget, but the game's win condition requires that all uncorrupted parties $k \in \mathcal{G} \setminus \mathcal{C}_{\mathsf{sas}}$ produce the same sas. If more than 2 of these parties run on different messages m, then the probability of this event will only further decrease, by the same argument as above. Overall, we have $\epsilon_2 \leq 1/2^d$.

5 Integrating SAS Authentication with X3DH

Role of X3DH in the Signal Protocol. An instantiation of our SAS-MMA protocol VrfAuth with ECVRF, denoted VrfAuth.ECVRF, can be integrated with Signal's X3DH protocol [6], to provide an additional layer of security against a compromised KDC server and compromised long-term identity keys. The Signal protocol consists of an Authenticated Key Exchange (AKE) protocol X3DH, which establishes a symmetric key between two interacting parties, Alice and Bob, followed by a "key-ratcheting" phase wherein Alice and Bob continually update their symmetric key with every message they send. We focus solely on the AKE protocol X3DH, and we will show how our SAS-MMA scheme VrfAuth (see Section 4) instantiated with ECVRF (see Section 2) can be integrated with X3DH to add additional layers of security to the AKE phase of Signal's protocol. (We omit Signal's key-ratcheting phase, as it is orthogonal to the changes we propose, and would execute the same way given the output of X3DH integrated with VrfAuth.) We believe that our method is generic and can be integrated with other AKE's, but we exemplify it for X3DH because of wide adoption and interest in X3DH and because our scheme can *re-use X3DH keys as VRF keys*, thus adding relatively low overhead to X3DH.

Protocol X3DH, shown in Figure 7, establishes an authenticated and secure symmetric key based on the assumption that the Signal server, denoted S in Fig. 7, supplies the correct counterparty's public key to each party, i.e. that S provides the true Bob's public key ipk^B to Alice and the true Alice's public key ipk^A to Bob. If the key server cannot be trusted then the protocol provides Alice and Bob no assurance that they really are talking to each other. In particular, a dishonest server might inject a public key of its own creation, which would allow it to impersonate users and establish communication sessions in their guise.

Security Without Trust in the Server. To address this risk, the Signal protocol provides a fingerprint mechanism for verifying public keys using trusted out-of-band communication. This mechanism uses QR codes which can be scanned and checked by Alice and Bob if they are physically collocated. The SAS-MMA protocol we propose offers an alternative out-of-band means for Alice and Bob

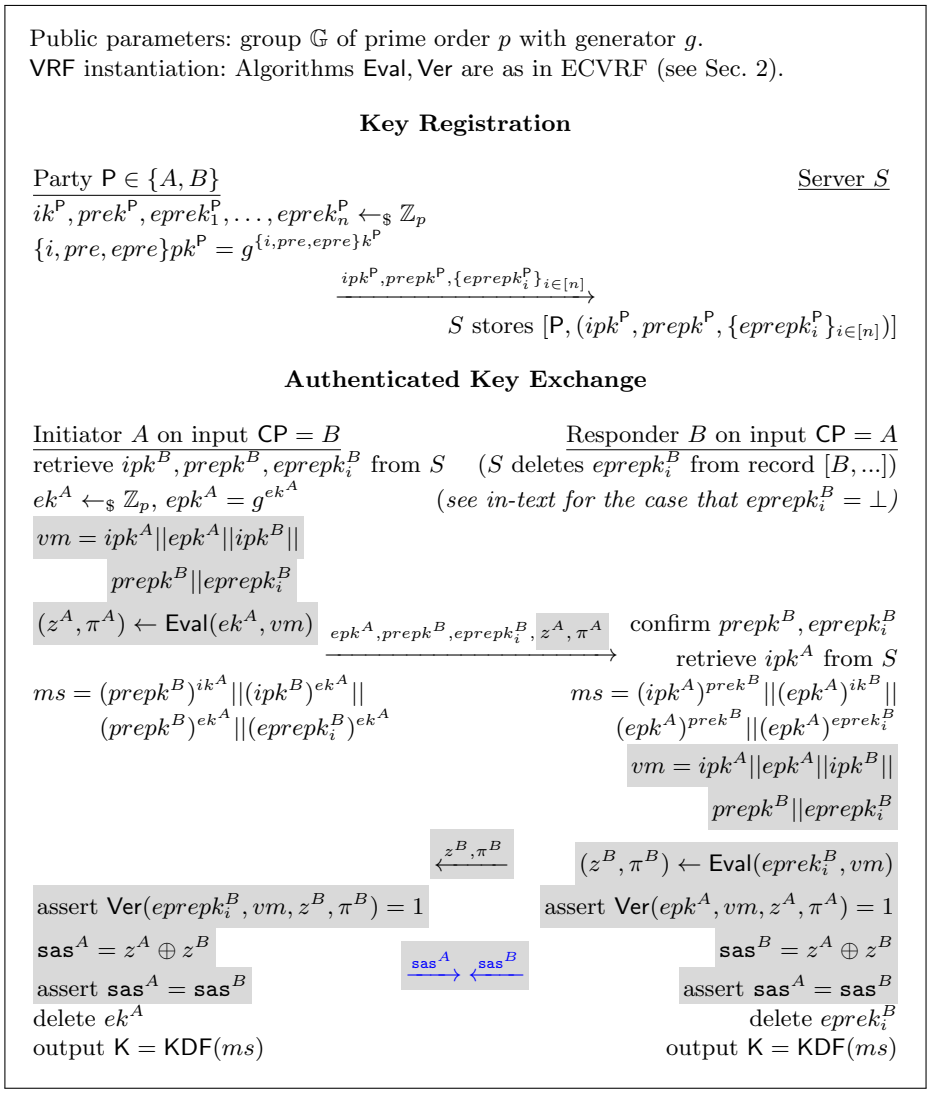

Fig. 7: X3DH protocol [6] amended by our SAS-MMA shown in gray. Communication over the out-of-band SAS channel is shown in blue.

to authenticate each other's keys (and/or the whole X3DH transcript) even if not physically collocated, assuming they have access to a *low-bandwidth* authenticated channel, i.e. in the *short authenticated string* (SAS) model.

If the Signal protocol is used to establish a secure voice connection then the d-bit SAS channel can be implemented by a trusted voice reading a d-bit `sas` value encoded as e.g. a decimal PIN, an alphanumerical string, or a phrase made of dictionary words from the user's designated language. Each party P

would reject the X3DH-established connection, which is represented by P's key output K in Figure 7, unless the SAS value $\mathtt{sas}^{\mathsf{CP}}$ value read off by their intended counterparty CP agrees with their own $\mathtt{sas}^{\mathsf{P}}$ value, and if they recognize the voice of their intended counterparty. If the protocol establishes a secure video connection then the view of a recognized face reading a matching SAS value provides an additional assurance, but in the video setting one can think of other SAS encodings e.g. as a sequence of gestures.

Integrating SAS-MMA with X3DH. Figure 7 presents *X3DH*, the AKE used in the Signal protocol, augmented by the SAS-MMA scheme $\mathsf{VrfAuth.ECVRF}$. The protocol assumes a group of prime order p with generator g. All key pairs used in X3DH have names of the form (xk, xpk), where the private key xk is chosen at random in $\mathbb{Z}_p$, the public key xpk is set as $xpk = g^{xk}$, and key name x ranges in the set $x \in \{i, pre, epre, e\}$.

In X3DH, parties first register with the key server by sending an identity public key ipk, a medium-term (also called "semi-static") public key $prepk$, and a list of ephemeral public keys $\{eprepk_i\}_{i\in[n]}$ for some n. A party's identity key is permanent, her medium-term key will be used in multiple key exchanges and periodically updated, and each ephemeral key is used in only a single key exchange. Key exchange begins when an initiator Alice requests a set of public keys, i.e. the identity key, the current medium-term key, and one ephemeral key, for a responder Bob from the key server. Alice also picks her own fresh ephemeral key ek^A and sends the corresponding public key epk^A to Bob along with her identity and pointers to Bob's medium-term and ephemeral keys she is using. Once Bob retrives Alice's identity key ipk^A from the key server, and retrieves his own medium-term and ephemeral private keys according to the received key pointers, both parties can combine these keys in a Diffie-Hellman manner to derive the same symmetric key K.

In parallel, Alice and Bob can use $\mathsf{VrfAuth.ECVRF}$ to authenticate their public keys and the X3DH session transcript. To confirm that the server is honest, it would suffice to SAS-authenticate the identity keys ipk^A and ipk^B. However, we include also the medium-term and ephemeral public keys in the SAS-MMA payload, to assure key exchange security even if either party's identity key is compromised. Since Alice and Bob SAS-authenticate the identity keys *and* the X3DH transcript, they will accept only (except for $1/2^d$ probability of SAS-MMA failure) if there is no active attack on this X3DH session. Consequently, key K will be secure even if parties' permanent keys ik^A and ik^B are compromised, as long as their ephemeral keys, resp. ek^A and $prek^B$ (or $eprek^B$) are not compromised.

Key Reuse. In order to maximize the ease of integration with existing X3DH deployments, we do not present a black-box composition of X3DH and our SAS-MMA $\mathsf{VrfAuth.ECVRF}$. Instead, we propose that the VRF keys used in $\mathsf{VrfAuth.ECVRF}$ are the same ephemeral keys that are already used in X3DH, resp. ek^A for Alice and $eprek_i^B$ for Bob. We thus avoid adding an extra round of vk exchanges to the protocol, and we avoid adding vk storage to the server. In particular, the protocol in Figure 7 involves no changes to the Signal key server, no changes to the key registration phase, and no changes to the com-

munication flow structure between Alice and Bob. Our SAS-MMA enhancement involves only an addition to Alice and Bob's client-side code and a small bandwidth overhead on the first two messages Alice and Bob exchange in the Signal protocol.

Since we opt for key-reuse rather than black-box composition, the security of the composed protocol in Figure 7 is not immediate. Nevertheless, in Theorems 3 and 4 below we argue that both X3DH and VrfAuth.ECVRF remain secure when their keys are reused in this way.

Running out of Ephemeral Keys. If S returns $eprepk_i^B = \perp$, i.e. if S runs out of Bob's ephemeral keys, protocol X3DH defaults to skipping the 4th term in the X3DH key derivation, i.e. term $(eprepk_i^B)^{ek^A} = (epk^A)^{eprek_i^B} = \mathsf{CDH}_g(epk^A, eprepk_i^B)$, effectively defaulting to 3DH AKE, with Bob's semi-static key pair $(prek^B, prepk^B)$ substituting for his ephemeral key.

However, in the proposed integration of X3DH and VrfAuth.ECVRF in Fig. 7, key pair $(eprek_i^B, eprepk_i^B)$ is used also for generating (z^B, π^B) by Bob and for verifying it by Alice. Therefore, in the case S runs out of Bob's ephemeral keys parties should default to using Bob's semi-static key pair $(prek^B, prepk^B)$ for this Bob-to-Alice VRF evaluation and verification. Indeed, to protect against the widest range of possible key compromise patterns, Alice could use a *combination* of her permanent and ephemeral keys, (ik^A, ek^A), in her VRF evaluation, and Bob could use a *combination* of his keys, $(ik^B, prek^B, eprek_i^B)$, in his VRF evaluation. We defer to Subsection 5.1 below for a simple method to generically combine several VRFs to obtain a hybrid VRF, secure as long as any of its component keys remains uncompromised.

Theorem 3. *The Signal protocol with* VrfAuth.ECVRF *integration (Figure 7) is a key-indistinguishable multi-stage key exchange protocol [6, Definition 4] under the GapDH assumption in ROM.*

Proof. To demonstrate the security of Signal, [6] define the notion of multi-stage key indistinguishability and prove that the Signal protocol satisfies it (under GapDH and in the ROM). The definition of this property is long and complex, involving many possible cases that arise when formalizing the Signal protocol's security goals of forward and future secrecy. We refer to [6, Definition 4] for the full details, and here we only use the multi-stage key indistinguishability game in a "black-box" way. Essentially, we only argue that the reuse of X3DH keys for ECVRF evaluation does not invalidate the existing security statement.

Consider a variation of the multi-stage key indistinguishability game where, after receiving any public key, the adversary $\mathcal{A}$ has oracle access to arbitrary ECVRF evaluations under it. Clearly, if the Signal protocol is secure in this game, then it is secure when its keys are reused as in Figure 7. Consider an adversary $\mathcal{A}$ with advantage ϵ against Signal in this modified game. We will construct adversary $\mathcal{A}'$ with advantage ϵ' against Signal in the plain multi-stage key indistinguishability game.

By the UC security of ECVRF (Theorem 1) there exists an efficient simulator $\mathsf{SIM}_{\mathsf{ECVRF}}$ such that no efficient environment can distinguish $\mathsf{SIM}_{\mathsf{ECVRF}}$ in inter-

action with $\mathcal{F}_{\mathsf{VRF}}$ from the real ECVRF protocol with probability greater than negligible $\epsilon_{\mathsf{ECVRF}}$ (under the CDH assumption and in the ROM). We therefore replace the ECVRF oracle access of $\mathcal{A}$ with this locally simulated access, i.e. $\mathcal{A}' = \mathcal{A} \leftrightarrow \mathsf{SIM}_{\mathsf{ECVRF}} \leftrightarrow \mathcal{F}_{\mathsf{VRF}}$. By inspection of $\mathsf{SIM}_{\mathsf{ECVRF}}$ [2, Section 9.1.3], this simulator never makes use of the ECVRF secret key until the time that that key is adaptively corrupted. We therefore need only slightly modify $\mathsf{SIM}_{\mathsf{ECVRF}}$ so that it receives its public and secret keys from the multi-stage key indistinguishability game rather than sampling them itself.

All together, we have $\epsilon \leq \epsilon' + \epsilon_{\mathsf{ECVRF}}$. Under GapDH and in the ROM, ϵ' is negligible per [6, Theorem 1] and $\epsilon_{\mathsf{ECVRF}}$ is negligible per Theorem 1. Therefore, ϵ is also negligible, hence the Signal protocol remains secure in spite of key reuse.

Theorem 4. VrfAuth *with Signal integration (Figure 7) is a secure SAS-MMA scheme (Definition 1) under the GapDH assumption in ROM.*

Proof. In this direction, we must decouple the Signal protocol from VrfAuth by showing that X3DH can be simulated without knowledge of one or more secret keys. To do so, we turn to a suggestion of [6] based on an idea of [8]. As seen in Figure 7, X3DH parties only use their secret keys to compute their public keys and to form the KDF input ms. If we model KDF as a random oracle, our simulator needs to be able to determine whether the group elements of ms are correctly formed for some session. If so, then KDF output must be programmed with the appropriate K; if not, then a random value should be programmed into KDF output instead. If the simulator holds the secret keys of honest parties, then this check can be easily performed by computing the Diffie-Hellman function just as an honest protocol participant does, but if the simulator has access to a DDH oracle then it can perform the same check using only the public keys.

Badertscher et al. [2] prove that ECVRF realizes $\mathcal{F}_{\mathsf{VRF}}$ under the CDH assumption in ROM. We must verify whether this statement remains true if ECVRF keys are reused in X3DH. Consider an environment $\mathcal{Z}$ interacting with this conjoined ECVRF + X3DH protocol which reuses keys. We move to a hybrid where the environment's X3DH interactions are instead answered by a simulator $\mathsf{SIM}_{\mathsf{X3DH}}$ (described in the previous paragraph) which has no access to the ECVRF secret keys but does have access to a DDH oracle $\mathcal{O}^{\mathsf{DDH}}$. We then apply the proof of ECVRF security given in [2, Section 9.1.3]. We refer the reader to [2] for the full proof and simply note by inspection that the only game hop affected by the environment's access to a DDH oracle is the one which reduces to CDH. Therefore by "upgrading" to a GapDH reduction, the proof is repaired, and we move to a hybrid where the real ECVRF protocol is replaced with $\mathcal{F}_{\mathsf{VRF}}$ and a simulator $\mathsf{SIM}_{\mathsf{ECVRF}}$. We can then combine simulator $\mathsf{SIM}_{\mathsf{ECVRF}}$ of [2] with simulator $\mathsf{SIM}_{\mathsf{X3DH}}$ in a single monolithic simulator SIM, which needs access to a DDH oracle. These 3 hybrids are depicted diagrammatically in Figure 8.

Overall, then, we have shown that the conjoined ECVRF + X3DH protocol is a secure VRF (i.e. it realizes $\mathcal{F}_{\mathsf{VRF}}$), but with the caveat that it might provide DDH oracle access to the environment of any higher-level protocol which uses it. However, it is easy to see that our SAS-MMA scheme VrfAuth remains secure in

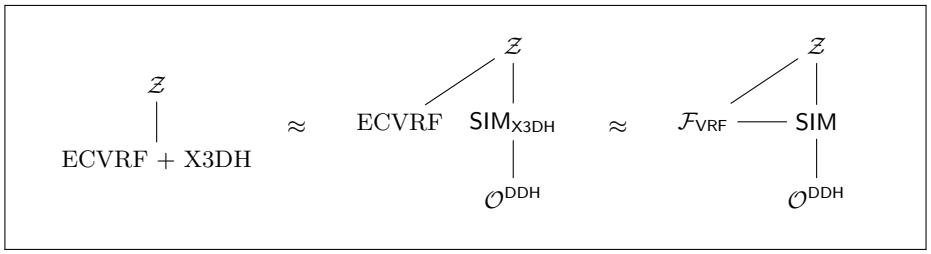

Fig. 8: Diagram of hybrids appearing in the proof of Theorem 4.

the presence of a DDH oracle, since it does not directly make use of any group elements. In other words, the proof of Theorem 2 goes through even if in the final game the adversary had an access to a DDH oracle. We conclude that VrfAuth is a secure SAS-MMA, under the GapDH assumption, in spite of key reuse.

5.1 Hybrid VRF Construction

Since in X3DH both parties use long-term and ephemeral keys, we can combine the VRFs defined by each such key to create a new VRF, which remains secure as long as either of the two keys remains uncompromised. Here, we briefly explain a "double VRF" construction, i.e. a VRF scheme DblVRF which composes two instances of VRF scheme VRF. (The construction can use two different VRF schemes, but we assume it is one scheme for the simplicity of notation, and because that will be the case in the application of VrfAuth.ECVRF to X3DH.) We will argue that if the underlying scheme VRF realizes the ideal UC functionality $\mathcal{F}_{\mathsf{VRF}}$, then so does the resulting "double VRF" scheme DblVRF, with an important caveat that the DblVRF is considered compromised only if the keys of *both* underlying VRF instances are compromised. In other words, the resulting VRF remains secure even if one of the two component keys is compromised.

We note that one can use the method below recursively, e.g. to support responder Bob using a combination of its *three* keys, $(ik^B, prek^B, eprek_i^B)$, as its VRF key. However, it is also clear that the scheme below can be generalized to combining any $n \geq 2$ keys in a straightforward way, without using recursion. We also note that the scheme described below is generic, and it is possible that one can optimize it, e.g. for the case where the underlying scheme VRF is ECVRF.

Given base scheme $\mathsf{VRF} = (\mathtt{ParG}, \mathsf{KG}, \mathsf{Eval}, \mathsf{Eval})$ with output length κ, the Double VRF scheme $\mathsf{DblVRF} = (\mathtt{ParG}', \mathsf{KG}', \mathsf{Eval}', \mathsf{Ver}')$ can be generically constructed as follows, given a hash function $H : \{0,1\}^* \to \{0,1\}^{p(\kappa)}$, modeled as a Random Oracle.

- $\mathtt{ParG}'(1^\kappa)$ generates $\mathsf{ppm} \leftarrow \mathtt{ParG}(1^\kappa)$.
- $\mathsf{KG}'(\mathsf{ppm})$ runs $\mathsf{KG}(\mathsf{ppm})$ twice to generate two key pairs (sk_1, vk_1) and (sk_2, vk_2), sets $sk \leftarrow (sk_1, sk_2)$ and $vk \leftarrow (vk_1, vk_2)$ and outputs (sk, vk).
- $\mathsf{Eval}'(sk, x)$ parses $(sk_1, sk_2) \leftarrow sk$, runs $(z_i, \pi_i) \leftarrow \mathsf{Eval}(sk_i, x)$ for $i = 1, 2$, sets $z \leftarrow H(x, vk_1, vk_2, z_1, z_2)$, $\pi \leftarrow (z_1, z_2, \pi_1, \pi_2)$, and outputs (z, π).

- $\mathsf{Ver}'(vk, x, z, \pi)$ parses $(vk_1, vk_2) \leftarrow vk$ and $(z_1, z_2, \pi_1, \pi_2) \leftarrow \pi$, and accepts if and only if $\mathsf{Ver}(vk_i, x, z_i, \pi_i) = 1$ for $i = 1, 2$ and $z = H(x, vk_1, vk_2, z_1, z_2)$.

Theorem 5. *If* VRF *realizes* $\mathcal{F}_{\mathsf{VRF}}$ *and hash* H *is a Random Oracle then* DblVRF *realizes* $\mathcal{F}_{\mathsf{VRF}}$*, where key* $vk = (vk_1, vk_2)$ *is not considered compromised until both* vk_1 *and* vk_2 *are compromised.*

Proof. (*sketch*) The proof is in a hybrid model where we assume that the base scheme VRF is replaced by ideal functionality $\mathcal{F}_{\mathsf{VRF}}$. For clarity we denote that instance of the functionality as $\mathcal{F}_{\mathsf{baseVRF}}$. We prove DblVRF security by providing a simulator $\mathcal{S}$ such that $\mathcal{S}$ in interaction with $\mathcal{F}_{\mathsf{VRF}}$ is indistinguishable from the real protocol DblVRF. $\mathcal{S}$ is not allowed to send $(\mathsf{Key\text{-}Comp}, \mathsf{P}_i, \mathsf{kptr})$ to $\mathcal{F}_{\mathsf{VRF}}$ until *both* of the $\mathcal{F}_{\mathsf{baseVRF}}$ keys corresponding to kptr have been compromised by the environment.

The simulator $\mathcal{S}$ runs the code of $\mathcal{F}_{\mathsf{baseVRF}}$, using its honest interfaces internally and exposing its adversarial interfaces to the environment. On key generation event $(\mathsf{KeyGen}, \mathsf{kptr}, \mathsf{P}_i)$ from $\mathcal{F}_{\mathsf{VRF}}$, $\mathcal{S}$ runs $\mathcal{F}_{\mathsf{baseVRF}}$ key generation twice as $(\mathsf{KeyGen}, \mathsf{kptr}||1)$ and $(\mathsf{KeyGen}, \mathsf{kptr}||2)$, receives vk_1 and vk_2, and returns $vk \leftarrow (vk_1, vk_2)$ to $\mathcal{F}_{\mathsf{VRF}}$. On evaluation event $(\mathsf{Eval}, \mathsf{P}_i, \mathsf{kptr}, x)$ from $\mathcal{F}_{\mathsf{VRF}}$, $\mathcal{S}$ runs $\mathcal{F}_{\mathsf{baseVRF}}$ evaluation twice as $(\mathsf{Eval}, \mathsf{kptr}||1, x)$ and $(\mathsf{Eval}, \mathsf{kptr}||2, x)$, receives (z_1, π_1) and (z_2, π_2), and returns $\pi \leftarrow (z_1, z_2, \pi_1, \pi_2)$ to $\mathcal{F}_{\mathsf{VRF}}$. On verification event $(\mathsf{Verify}, vk, x, z, \pi, S_{\mathsf{Eval}})$ from $\mathcal{F}_{\mathsf{VRF}}$, $\mathcal{S}$ parses $(vk_1, vk_2) \leftarrow vk$ and $(z_1, z_2, \pi_1, \pi_2) \leftarrow \pi$, runs $\mathcal{F}_{\mathsf{baseVRF}}$ verification twice as $(\mathsf{Verify}, vk_1, x, z_1, \pi_1)$ and $(\mathsf{Verify}, vk_2, x, z_2, \pi_2)$, receives f_1 and f_2, and returns $\phi \leftarrow f_1 \wedge f_2$ to $\mathcal{F}_{\mathsf{VRF}}$.

$\mathcal{S}$ must also program responses to adversarial random oracle queries of the form $H(x, vk_1, vk_2, z_1, z_2)$. To do so, $\mathcal{S}$ first accesses its internal T_{base} table to check whether $T_{\mathsf{base}}[vk_1, x] = z_1$ and $T_{\mathsf{base}}[vk_2, x] = z_2$. If not, $\mathcal{S}$ programs H with a random value. If yes, then $\mathcal{S}$ must program H in agreement with the honest interface of $\mathcal{F}_{\mathsf{VRF}}$. If $vk \leftarrow (vk_1, vk_2)$ has not yet been registered with $\mathcal{F}_{\mathsf{VRF}}$, then $\mathcal{S}$ uses $(\mathsf{MalKeyGen}, vk)$ to register it as a new adversarial key. If instead it is already registered with $\mathcal{F}_{\mathsf{VRF}}$ as an uncompromised key $vk \notin \mathcal{C}$ owned by P_i with pointer kptr, then $\mathcal{S}$ uses (PastEvaluations) and checks whether $(vk, x, z) \in S_{\mathsf{Eval}}$. If not, then both vk_1 and vk_2 *must be compromised* in $\mathcal{F}_{\mathsf{baseVRF}}$; otherwise, $T_{\mathsf{base}}[vk_1, x]$ and $T_{\mathsf{base}}[vk_2, x]$ would not be defined. Therefore, $\mathcal{S}$ is allowed to send $(\mathsf{Key\text{-}Comp}, \mathsf{P}_i, \mathsf{kptr})$ to $\mathcal{F}_{\mathsf{VRF}}$. Now, in all cases, $\mathcal{S}$ can successfully query $z \leftarrow (\mathsf{MalEval}, vk, x)$ from $\mathcal{F}_{\mathsf{VRF}}$ and program that value to H.

This simulation strategy fails in the case that $T_{\mathsf{base}}[vk_1, x]$ and $T_{\mathsf{base}}[vk_2, x]$ are randomly sampled as exactly z_1 and z_2 sometime after random oracle query $H(x, vk_1, vk_2, z_1, z_2)$. Because DblVRF uses z_1 and z_2 of length κ, this event occurs with negligible probability.

6 Conclusion

In summary, we propose a new Short Authenticated String Message Authentication (SAS-MA) scheme using Verifiable Random Functions (VRF) which allows

for key reuse. Given any (Authenticated) Key Exchange scheme we can compose our SAS-MA to authenticate its transcript to get AKE which is secure against active attacks assuming users have access to an authenticated but short bandwidth channel. We study the specific but popular use case of X3DH AKE, and we argue that the same keys used within X3DH can be used for our VRF-based SAS-MA. The combination of X3DH execution and SAS-authentication of X3DH transcripts forms AKE without reliance on a Public Key Infrastructure (PKI) or a trusted Key Distribution Center (KDC).

References

1. Christian Badertscher, Peter Gazi, Aggelos Kiayias, Alexander Russell, and Vassilis Zikas. Ouroboros genesis: Composable proof-of-stake blockchains with dynamic availability. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018*, pages 913–930, Toronto, ON, Canada, October 15–19, 2018. ACM Press.
2. Christian Badertscher, Peter Gaži, Iñigo Querejeta-Azurmendi, and Alexander Russell. On UC-secure range extension and batch verification for ECVRF. *Cryptology ePrint Archive*, 2022.
3. Dirk Balfanz, Diana Smetters, Paul Stewart, and H. Chi Wong. Talking to strangers: Authentication in ad-hoc wireless networks. In *Network and Distributed System Security Symposium (NDSS)*, 2002.
4. Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145, Las Vegas, NV, USA, October 14–17, 2001. IEEE Computer Society Press.
5. Melissa Chase and Anna Lysyanskaya. Simulatable vrfs with applications to multi-theorem NIZK. In *Annual International Cryptology Conference*, pages 303–322. Springer, 2007.
6. Katriel Cohn-Gordon, Cas Cremers, Benjamin Dowling, Luke Garratt, and Douglas Stebila. A formal security analysis of the Signal messaging protocol. In *2017 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 451–466, 2017.
7. Bernardo David, Peter Gazi, Aggelos Kiayias, and Alexander Russell. Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part II*, volume 10821 of *LNCS*, pages 66–98, Tel Aviv, Israel, April 29 – May 3, 2018. Springer, Cham, Switzerland.
8. Jean Paul Degabriele, Anja Lehmann, Kenneth G. Paterson, Nigel P. Smart, and Mario Strefler. On the joint security of encryption and signature in emv. In Orr Dunkelman, editor, *Topics in Cryptology – CT-RSA 2012*, pages 116–135, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
9. Christian Gehrmann, Chris J. Mitchell, and Kaisa Nyberg. Manual authentication for wireless devices. *RSA CryptoBytes*, 7(1):24–37, 2004.
10. Sharon Goldberg, Leonid Reyzin, Dimitrios Papadopoulos, and Jan Vcelak. Verifiable random functions (VRFs). Internet-Draft, IRTF, 202. `https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-vrf-14`.
11. Michael T. Goodrich, Michael Sirivianos, John Solis, Gene Tsudik, and Ersin Uzun. Loud and clear: Human-veriable authentication based on audio. In *International Conference on Distributed Computing Systems (ICDCS)*, 2006.

12. Yanqi Gu, Stanislaw Jarecki, Phillip Nazarian, and Apurva Rai. Security without trusted third parties: VRF-based authentication with short authenticated strings. In *Advances in Cryptology — ASIACRYPT*. Springer, 2025.
13. Stanisław Jarecki and Nitesh Saxena. Authenticated key agreement with key reuse in the short authenticated strings model. In *Security and Cryptography for Networks: 7th International Conference (SCN)*, pages 253–270. Springer, 2010.
14. Sven Laur, N. Asokan, and Kaisa Nyberg. Efficient mutual data authentication using manually authenticated strings. Cryptology ePrint Archive, Paper 2005/424, 2005.
15. Sven Laur and Kaisa Nyberg. Efficient mutual data authentication using manually authenticated strings. In *International Conference on Cryptology and Network Security (ICNS)*, pages 90–107. Springer, 2006.
16. Sven Laur and Sylvain Pasini. SAS-based group authentication and key agreement protocols. In *Public Key Cryptography (PKC)*, pages 197–213. Springer, 2008.
17. Silvio Micali, Michael Rabin, and Salil Vadhan. Verifiable random functions. In *40th annual symposium on foundations of computer science (cat. No. 99CB37039)*, pages 120–130. IEEE, 1999.
18. David Naccache, David M'RaÏhi, Serge Vaudenay, and Dan Raphaeli. Can dsa be improved?—complexity trade-offs with the digital signature standard—. In *Advances in Cryptology—EUROCRYPT'94: Workshop on the Theory and Application of Cryptographic Techniques Perugia, Italy, May 9–12, 1994 Proceedings 13*, pages 77–85. Springer, 1995.
19. Dimitrios Papadopoulos, Duane Wessels, Shumon Huque, Moni Naor, Jan Včelák, Leonid Reyzin, and Sharon Goldberg. Making NSEC5 practical for DNSSEC. *Cryptology ePrint Archive*, 2017.
20. Sylvain Pasini and Serge Vaudenay. SAS-based authenticated key agreement. In *International Workshop on Public Key Cryptography (PKC)*, pages 395–409. Springer, 2006.
21. Chris Peikert and Jiayu Xu. Classical and quantum security of elliptic curve vrf, via relative indifferentiability. In *Cryptographers' Track at the RSA Conference*, pages 84–112. Springer, 2023.
22. Christopher Portmann. Key recycling in authentication. *IEEE Transactions on Information Theory*, 60(7):4383–4396, 2014.
23. Ramnath Prasad and Nitesh Saxena. Efficicient device pairing using human-comparable synchronized audiovisual patterns. In *Applied Cryptography and Network Security (ACNS)*, 2008.
24. Claudio Soriente, Gene Tsudik, and Ersin Uzun. BEDA: Button-enabled device association. In *International Workshop on Security for Spontaneous Interaction (IWSSI)*, 2007.
25. Serge Vaudenay. Secure communications over insecure channels based on short authenticated strings. In *Annual International Cryptology Conference (Crypto)*, pages 309–326. Springer, 2005.
26. Rupeng Yang, Man Ho Au, Zhenfei Zhang, Qiuliang Xu, Zuoxia Yu, and William Whyte. Efficient lattice-based zero-knowledge arguments with standard soundness: Construction and applications. In *Advances in Cryptology – CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part I*, page 147–175, Berlin, Heidelberg, 2019. Springer-Verlag.

A SAS-MMA Correctness

The correctness of a SAS-MMA scheme Π is formally defined via game $\mathsf{Game}^{\mathsf{MMAcor}}_{\mathcal{A},\Pi}$ in Figure 9, where the adversary can initialize an arbitrary number of keys for any party and run SAS-MMA protocol instances for any message and any set of parties and keys these parties hold.

In the correctness game each party P_i can create multiple keys, and the adversary can choose an arbitrary set of $n \geq 2$ parties and the keys they should use in the protocol. Moreover, key generation and SAS-MMA execution queries can be arbitrarily interleaved, and at key generation we leak each private key to the adversary, which implies that correctness is not affected by key compromise.

$\underline{\mathsf{Game}^{\mathsf{MMAcor}}_{\mathcal{A},\Pi}(\kappa)}$:
$\forall i, \mathsf{sid}$ set $\mathsf{klist}[i] \leftarrow \varnothing$ and $\mathsf{accept}[i, \mathsf{sid}] \leftarrow 1$
$(i, \mathsf{sid}) \leftarrow \mathcal{A}^{\mathsf{Init},\mathsf{Exec}}(1^\kappa)$
if $\mathsf{accept}[i, \mathsf{sid}] = 0$ return 0 else return 1

$\underline{\mathsf{Init}(i)}$: *(can be called multiple times for each party P_i)*
$(sk, vk) \leftarrow \mathsf{Init}(1^\kappa)$, $\mathsf{klist}[i] \leftarrow \mathsf{klist}[i] \cup \{(sk, vk)\}$, return (sk, vk)

$\underline{\mathsf{Exec}(\mathsf{sid}, n, \sigma, \mathcal{G}, \mathsf{VK}, m)}$:
assert $|\mathcal{G}| = n$ and $\sigma : [n] \xrightarrow{1\text{–}1} \mathcal{G}$ *(σ is an ordering of parties in group $\mathcal{G}$)*
assert $\exists\{(sk_t, vk_t)\}_{t\in[n]}$ s.t. $\mathsf{VK} = (vk_1, ..., vk_n) \ \wedge \ \forall t \in [n] \ (sk_t, vk_t) \in \mathsf{klist}[\sigma(t)]$
for all $t \in [n]$ set $pm_t \leftarrow \mathsf{MsgGen}(sk_t, \mathsf{VK}, m)$, set $\mathsf{PM} = (pm_1, ..., pm_n)$
for all $t \in [n]$ set $\mathtt{sas}_t \leftarrow \mathsf{SasGen}(sk_t, \mathsf{VK}, m, \mathsf{PM})$, set $\mathsf{SAS} = (\mathtt{sas}_1, ..., \mathtt{sas}_n)$
for all $t \in [n]$ set $\mathsf{accept}[\sigma(t), \mathsf{sid}] \leftarrow 0$ if $\exists j \in [n]$ s.t. $\mathtt{sas}_j \neq \mathtt{sas}_i$
return $(\mathsf{PM}, \mathsf{SAS})$

Fig. 9: Correctness game for SAS-MMA scheme $\Pi = (\mathsf{Init}, \mathsf{MsgGen}, \mathsf{SasGen})$

In game $\mathsf{Game}^{\mathsf{MMAcor}}_{\mathcal{A},\Pi}$, protocol participants are defined by set $\mathcal{G} = \{i_1, ..., i_n\}$ of identifiers, and $\sigma : [n] \to \mathcal{G}$ is their ordering, e.g. $\sigma(t) = i_t$. Note that algorithms MsgGen and SasGen take as inputs vectors of resp. keys VK and protocol messages PM, which implies that parties must use a common ordering of all participants, e.g. via a lexicographic ordering of their identifiers.

Definition 2. *A SAS-MMA scheme Π is* correct *if for all efficient algorithms $\mathcal{A}$ there exists a negligible function ϵ s.t.* $\Pr[0 \leftarrow \mathsf{Game}^{\mathsf{MMAcor}}_{\mathcal{A},\Pi}(1^\kappa)] \leq \epsilon(\kappa)$.

B Verifiable Random Function Game-Based Properties

As explained in Section 2, a VRF scheme must satisfy *correctness/completeness*, *uniqueness/soundness*, and *(simulatable) pseudorandomness*.[9] Chase and

[9] We include these game-based definitions for reference, but they are all implied by the UC VRF notion defined in Section 2, Figure 1.

Lysysanskaya [5] extended VRF pseudorandomness to *simulatability*, which models the zero-knowledge property of proofs π output by Eval. This is formally captured by experiment $\mathsf{Game}^{\mathsf{ZK\text{-}PRF}}_{\mathcal{A},\mathsf{VRF},\mathcal{S}}$ shown in Figure 10: No efficient adversary can distinguish between oracle $\mathsf{Eval}_{sk}(\cdot)$ and an oracle which sets $z = R(x)$ where R is a random function while π is set by a simulator $\mathcal{S}$ on input (vk, x, z) and a trapdoor td generated together with global parameters ppm.

$\mathsf{Game}^{\mathsf{SND}}_{\mathcal{A},\mathsf{VRF}}(1^\kappa)$:

$\mathsf{ppm} \leftarrow \mathtt{ParG}(1^\kappa)$

$(vk, x, z_0, z_1, \pi_0, \pi_1) \leftarrow \mathcal{A}(\mathsf{ppm})$

return $(z_0 \neq z_1)$

$\quad \wedge\ \mathsf{Ver}(vk, x, z_0, \pi_0) = 1$

$\quad \wedge\ \mathsf{Ver}(vk, x, z_1, \pi_1) = 1$

Oracle $\mathsf{TestEval}[\mathsf{td}, sk, vk, b](x)$:

if $(b == 0)$ $(z, \pi) \leftarrow \mathsf{Eval}(sk, x)$

else $z \leftarrow \mathsf{R}(x)$

$\quad \pi \leftarrow \mathcal{S}.\mathsf{Eval}(\mathsf{td}, vk, x, z)$

return(y, π)

$\mathsf{Game}^{\mathsf{ZK\text{-}PRF}}_{\mathcal{A},\mathsf{VRF},\mathcal{S}}(1^\kappa)$:

$b \leftarrow_\$ \{0,1\}$

if $(b == 0)$

$\quad \mathsf{ppm} \leftarrow \mathtt{ParG}(1^\kappa), \mathsf{td} \leftarrow \perp$

else $(\mathsf{ppm}, \mathsf{td}) \leftarrow \mathcal{S}.\mathtt{ParG}(1^\kappa)$

$\quad \mathsf{R} \leftarrow_\$ \mathcal{F}[\{0,1\}^*, \{0,1\}^{p(\kappa)}]$

$(sk, vk) \leftarrow \mathsf{KG}(\mathsf{ppm})$

$b' \leftarrow \mathcal{A}^{\mathsf{TestEval}[\mathsf{td}, sk, vk, b](\cdot)}(\mathsf{ppm}, vk)$

return $(b' == b)$

Notation: $\mathcal{F}[X, Y]$ is a set of all functions from domain X to range Y

Fig. 10: Security games for VRF *uniqueness* and *pseudorandomness*

- A VRF scheme is *correct*, a.k.a. *complete*, if for all x the probability that $\mathsf{Ver}(vk, x, z, \pi) = 0$ for $\mathsf{ppm} \leftarrow \mathtt{ParG}(1^\kappa)$, $(sk, vk) \leftarrow \mathsf{KG}(\mathsf{ppm})$, and $(z, \pi) \leftarrow \mathsf{Eval}(sk, x)$, is a negligible function of the security parameter κ.[10]
- A VRF scheme is *unique*, a.k.a. *sound*, if for all efficient algorithms $\mathcal{A}$ there is a negligible function ϵ s.t. $\Pr[1 \leftarrow \mathsf{Game}^{\mathsf{SND}}_{\mathcal{A},\mathsf{VRF}}(1^\kappa)] \leq \epsilon(\kappa)$, for $\mathsf{Game}^{\mathsf{SND}}_{\mathcal{A},\mathsf{VRF}}$ defined in Figure 10.
- A VRF scheme is *pseudorandom* (and *simulatable*) if there exists an efficient $\mathcal{S}$ s.t. for all efficient $\mathcal{A}$ there is a negligible function ϵ s.t. $\Pr[1 \leftarrow \mathsf{Game}^{\mathsf{ZK\text{-}PRF}}_{\mathcal{A},\mathsf{VRF},\mathcal{S}}(1^\kappa)] \leq \frac{1}{2} + \epsilon(\kappa)$, for $\mathsf{Game}^{\mathsf{ZK\text{-}PRF}}_{\mathcal{A},\mathsf{VRF},\mathcal{S}}$ defined in Figure 10.

[10] Allowing negligible correctness error accomodates e.g. lattice-based VRF's [26].